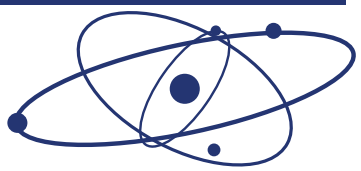
Structure-Preserving Integration of the Gravitational N-Body Problem

From explicit Euler to high-order symplectic composition methods
Computational physics project by Eren Can
Revised version: November 09, 2026

Darmstadt



Course project
Computational Physics
Winter term 2024/25

Revised edition 2026

Abstract

The gravitational N-body problem has no closed-form solution for $N \geq 3$, so its long-term behaviour can only be explored numerically. Over long times, though, the main danger is not the local truncation error of the integrator. It is the slow, systematic violation of the geometric structure of Hamiltonian dynamics: energy drifts, orbits spiral, and phase errors grow quadratically.

This thesis studies this problem systematically. Starting from Newton's law of gravitation, we derive the Hamiltonian formulation, the conservation laws and the exact solution of the two-body problem. We then develop a hierarchy of integrators of increasing sophistication, which runs from the explicit and symplectic Euler methods through Störmer–Verlet (leapfrog) and the classical Runge–Kutta method to Yoshida's fourth- and sixth-order symplectic composition methods and an adaptive Dormand–Prince pair. For each method we derive the order of accuracy, the linear stability limit and whether it preserves the symplectic structure, and we use backward error analysis to explain why symplectic methods keep the energy error bounded.

All methods are implemented in a modular, tested Python package. It uses vectorised force kernels, a common integrator interface with a registry, a reproducible experiment pipeline and an interactive viewer whose overlays show past trails and predicted future trajectories. Nine experiments evaluate the implementation against the theory. The measured convergence orders agree with the theoretical ones to within 0.3, and symplectic methods show linear instead of quadratic error growth. A Solar System simulation over 1000 years keeps the relative energy error below $3 \cdot 10^{-6}$ at one force evaluation per simulated day. Close encounters in the Pythagorean three-body problem require adaptive time steps spanning six orders of magnitude. Finally, the energy fluctuation reported in the original course project turns out to be mostly an artefact of an inconsistent energy diagnostic.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Origin and scope of this work . . . 2
- 1.3 Contributions . . . 2
- 1.4 Outline . . . 3

2 Physical Background **4**
- 2.1 Newtonian gravitation and equations of motion . . . 4
- 2.2 Hamiltonian formulation . . . 5
- 2.3 Conservation laws . . . 6
- 2.4 The two-body problem . . . 7
- 2.5 Three and more bodies: non-integrability and chaos . . . 9
- 2.6 Gravitational softening . . . 10
- 2.7 Units and scaling . . . 11

3 Numerical Integration of Hamiltonian Systems **12**
- 3.1 One-step methods, order and cost . . . 12
- 3.2 Explicit Euler method . . . 13
- 3.3 Symplectic Euler method . . . 13
- 3.4 Störmer–Verlet / leapfrog . . . 15
- 3.5 Backward error analysis and long-time behaviour . . . 16
- 3.6 Runge–Kutta methods . . . 17
- 3.7 Composition methods . . . 18
- 3.8 Adaptive step-size control . . . 19
- 3.9 Summary of the methods . . . 20

4 Implementation **21**
- 4.1 Repository structure . . . 21
- 4.2 Architecture . . . 22

4.3 Data model . . . 23
4.4 Vectorised force evaluation . . . 24
4.5 Integrators . . . 25
4.6 Simulation driver and trajectories . . . 26
4.7 Scenarios . . . 26
4.8 Interactive viewer and trajectory overlays . . . 27
4.9 Testing and reproducibility . . . 28
4.10 Improvements over the original implementation . . . 30

5 Evaluation **31**
5.1 Metrics and methodology . . . 31
5.2 Linear stability and phase space . . . 32
5.3 Convergence order and work-precision . . . 33
5.4 Long-term behaviour at equal cost . . . 35
5.5 Revisiting the original experiment . . . 37
5.6 Periodic three-body orbit: the figure-eight . . . 38
5.7 Chaos and close encounters: the Pythagorean problem . . . 39
5.8 The Solar System over 1000 years . . . 42
5.9 Performance . . . 44
5.10 Discussion . . . 44

6 Conclusion and Outlook **46**
6.1 Summary . . . 46
6.2 Recommendations . . . 47
6.3 Limitations and outlook . . . 47

A User Guide **49**
A.1 Installation . . . 49
A.2 Command-line interface . . . 49
A.3 Python API . . . 50
A.4 Scenario file format . . . 50
A.5 Reproducing the results . . . 51

B Method Coefficients **52**
B.1 Composition weights . . . 52
B.2 Dormand–Prince 5(4) . . . 52

Notation

Physical quantities

N	number of bodies
d	spatial dimension ($d = 2$ in all experiments, the code supports $d = 3$)
m_i, M	mass of body i, total mass $M = \sum_i m_i$
$\boldsymbol{r}_i, \boldsymbol{v}_i, \boldsymbol{a}_i$	position, velocity and acceleration of body i
$\boldsymbol{p}_i$	momentum $\boldsymbol{p}_i = m_i \boldsymbol{v}_i$
$\boldsymbol{r}_{ij}$	separation vector $\boldsymbol{r}_{ij} = \boldsymbol{r}_j - \boldsymbol{r}_i$, $r_{ij} = \lvert\boldsymbol{r}_{ij}\rvert$
G	gravitational constant (unit dependent, e.g. $G = 1$ or $G = 4\pi^2 \mathrm{AU}^3 M_\odot^{-1} \mathrm{yr}^{-2}$)
ε	Plummer softening length
T, V, H	kinetic energy, potential energy, Hamiltonian $H = T + V$
E	total energy (value of H along a solution)
$\boldsymbol{P}, \boldsymbol{L}, \boldsymbol{R}$	total linear momentum, angular momentum, centre of mass
a, e	semi-major axis and eccentricity of a Kepler orbit
P_{orb}	orbital period; $n = 2\pi / P_{\mathrm{orb}}$ mean motion
μ	gravitational parameter $\mu = G(m_1 + m_2)$ of a two-body problem

Numerical quantities

$\boldsymbol{y}$	phase-space vector $\boldsymbol{y} = (\boldsymbol{q}, \boldsymbol{p})$ or $(\boldsymbol{x}, \boldsymbol{v})$
h	time step (also Δt in the code: `dt`)
$t_n, \boldsymbol{y}_n$	discrete time $t_n = nh$ and numerical approximation of $\boldsymbol{y}(t_n)$
φ_t	exact flow of the differential equation
Φ_h	one-step map of a numerical method
p	order of accuracy (global error $\mathcal{O}(h^p)$)
J	canonical symplectic matrix $J = \left(\begin{smallmatrix} 0 & I \\ -I & 0 \end{smallmatrix}\right)$
$\tilde{H}$	modified Hamiltonian of a symplectic method
$\delta E(t)$	relative energy error $(E(t) - E_0) / \lvert E_0\rvert$
γ_k	weights of a composition method
λ	(finite-time) Lyapunov exponent

1 Introduction

1.1 Motivation

The gravitational N-body problem asks for the motion of N point masses that attract each other according to Newton's law of gravitation. It is one of the oldest problems of mathematical physics and is still central to modern astrophysics: planetary systems, star clusters, galaxies and the large-scale structure of the universe are all modelled as gravitational N-body systems [1].

For $N = 2$ the problem is integrable and is solved by Kepler's laws. For $N \geq 3$ no general closed-form solution exists. Poincaré showed that the three-body problem has, in general, no analytic first integrals beyond the classical ones and that its dynamics can be chaotic [2]. Sundman's convergent series solution [3] exists in principle but converges far too slowly to be of practical use. Numerical integration is therefore the method of choice.

Integrating the equations of motion numerically for a short time is easy. Integrating them faithfully over many orbital periods is not. Every numerical method introduces a small error in each step, and **how** these errors accumulate matters more than how large they are. A method can have a tiny local error and still let the total energy drift steadily, so that a planet slowly spirals into its star. Another method with a larger local error can keep the energy error bounded forever. The difference comes from the **geometric structure** of Hamiltonian dynamics, and in particular from the symplecticity of the flow [4], [5]. Understanding this structure and choosing algorithms that respect it is the subject of *geometric numerical integration* and the central theme of this thesis.

1.2 Origin and scope of this work

This thesis is a revised and substantially extended version of a course project carried out in the course *Computational Physics* (winter term 2024/25) together with Chahid El Aaouadi and Simon Pfeiffer [6]. The original project implemented a two-dimensional N-body simulation with a leapfrog integrator and a real-time visualisation, and it assessed the quality of the simulation through the ratio $|E_t/E_0|$ of the current to the initial total energy.

The present work keeps the physical problem but rebuilds the project from the ground up with three goals:

1. **Theory.** Give a self-contained derivation of the physics (Hamiltonian formulation, conservation laws, exact two-body solution, softening, chaos) and of the numerical methods, ordered by increasing sophistication, including their order, stability and structure-preserving properties.
2. **Implementation.** Provide a clean, modular and efficient implementation in which new integrators, force models and scenarios can be added without touching existing code, and in which every theoretical claim is backed by an automated test.
3. **Evaluation.** Quantify the behaviour of all methods with well-defined metrics (energy, momentum and angular-momentum errors, global errors against exact or high-precision references, empirical convergence orders, time-reversibility, physical observables, computational cost) on a set of benchmark problems of increasing difficulty.

1.3 Contributions

Compared with the original course project, the main contributions are:

- a hierarchy of nine integrators, from explicit Euler to sixth-order symplectic composition methods and an adaptive Dormand–Prince pair, all behind a common interface and a registry;
- derivations of the order conditions, stability limits, modified energies and error-growth laws that explain the observed behaviour;

- vectorised force kernels that are up to two orders of magnitude faster than the original pair loop, and a leapfrog that reuses the last force evaluation and so needs half as many evaluations per step;
- a consistent energy diagnostic for softened gravity. The re-evaluation in Section 5.5 shows that most of the energy fluctuation reported in the original project was an artefact of an inconsistent potential;
- an evaluation framework with nine reproducible experiments whose figures and tables are generated automatically and read directly by this document;
- an interactive viewer with two trajectory overlays: fading trails of the past motion, and forecasts of the future motion computed by integrating a “ghost” copy of the system.

1.4 Outline

Section 2 introduces the physics of the gravitational N-body problem, from Newton’s law to the Hamiltonian formulation, the conservation laws and the exact solution of the Kepler problem. Section 3 develops the numerical methods step by step and analyses their accuracy, stability and geometric properties. Section 4 describes the software architecture and the most important implementation decisions. Section 5 defines the evaluation metrics and presents the experiments. Section 6 summarises the results and discusses possible extensions. The appendices contain a short user guide and the coefficients of all methods.

2 Physical Background

This chapter collects the physics that the simulation implements and that the evaluation relies on. We start from Newton's law of gravitation (Section 2.1), reformulate it in Hamiltonian form (Section 2.2), derive the conservation laws used as accuracy diagnostics (Section 2.3), solve the two-body problem exactly to obtain a reference solution (Section 2.4), discuss what changes for $N \geq 3$ (Section 2.5), and finally treat softening (Section 2.6) and units (Section 2.7).

2.1 Newtonian gravitation and equations of motion

Consider N point masses $m_i > 0$ at positions $\boldsymbol{r}_i \in \mathbb{R}^d$, $i = 1, \dots, N$. By Newton's law of gravitation, body j exerts on body i the force

$$\boldsymbol{F}_{ij} = G \frac{m_i m_j}{r_{ij}^3} \boldsymbol{r}_{ij}, \quad \boldsymbol{r}_{ij} = \boldsymbol{r}_j - \boldsymbol{r}_i, \quad r_{ij} = |\boldsymbol{r}_{ij}|, \tag{1}$$

directed from i towards j (Figure 2.1). The force is antisymmetric, $\boldsymbol{F}_{ji} = -\boldsymbol{F}_{ij}$ (Newton's third law), and **central**, i.e. parallel to the line connecting the bodies. Gravitational forces superpose linearly, so Newton's second law $m_i \ddot{\boldsymbol{r}}_i = \sum_{j \neq i} \boldsymbol{F}_{ij}$ yields the equations of motion

$$\ddot{\boldsymbol{r}}_i = \boldsymbol{a}_i(\boldsymbol{r}_1, \dots, \boldsymbol{r}_N) = G \sum_{j \neq i} m_j \frac{\boldsymbol{r}_j - \boldsymbol{r}_i}{|\boldsymbol{r}_j - \boldsymbol{r}_i|^3}, \quad i = 1, \dots, N. \tag{2}$$

These are dN coupled, nonlinear second-order ordinary differential equations (ODEs), or $2dN$ first-order ones for positions and velocities. The right-hand side is singular at collisions $r_{ij} \to 0$, and evaluating it costs $\mathcal{O}(N^2)$ operations because every pair of bodies interacts.

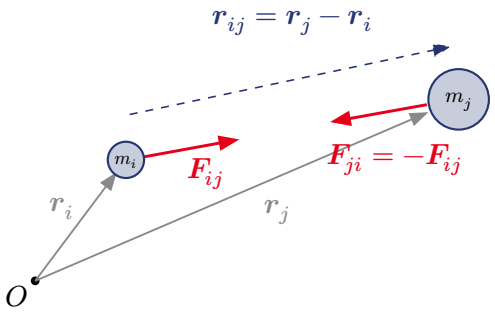


Figure 2.1: Two bodies exert equal and opposite central forces on each other. The force on body i points along the separation vector $\boldsymbol{r}_{ij}$.

2.2 Hamiltonian formulation

The geometric properties that distinguish good integrators from bad ones are formulated most naturally in the Hamiltonian picture [4], [7]. Collect the positions in $\boldsymbol{q} = (\boldsymbol{r}_1, \ldots, \boldsymbol{r}_N) \in \mathbb{R}^{dN}$ and the momenta $\boldsymbol{p}_i = m_i \dot{\boldsymbol{r}}_i$ in $\boldsymbol{p} \in \mathbb{R}^{dN}$. The Hamiltonian is the total energy

$$H(\boldsymbol{q}, \boldsymbol{p}) = \underbrace{\sum_{i=1}^{N} \frac{|\boldsymbol{p}_i|^2}{2m_i}}_{T(\boldsymbol{p})} \underbrace{- \sum_{i<j} \frac{G m_i m_j}{r_{ij}}}_{V(\boldsymbol{q})}, \tag{3}$$

and the equations of motion are Hamilton's equations

$$\dot{\boldsymbol{q}} = \frac{\partial H}{\partial \boldsymbol{p}} = M^{-1}\boldsymbol{p}, \quad \dot{\boldsymbol{p}} = -\frac{\partial H}{\partial \boldsymbol{q}} = -\nabla V(\boldsymbol{q}), \tag{4}$$

with the diagonal mass matrix M. One checks that $-\partial V / \partial \boldsymbol{r}_i = \sum_{j \neq i} \boldsymbol{F}_{ij}$, so Equation 4 is equivalent to Equation 2. With the phase-space vector $\boldsymbol{y} = (\boldsymbol{q}, \boldsymbol{p}) \in \mathbb{R}^{2dN}$ and the canonical matrix

$$J = \begin{pmatrix} 0 & I \\ -I & 0 \end{pmatrix}, \quad J^T = J^{-1} = -J, \tag{5}$$

Hamilton's equations take the compact form $\dot{\boldsymbol{y}} = J \nabla H(\boldsymbol{y})$.

Two structural features of Equation 3 are exploited by the integrators of Section 3:

- **Separability.** $H = T(\boldsymbol{p}) + V(\boldsymbol{q})$. The sub-systems with Hamiltonian T alone ("drift": $\dot{\boldsymbol{q}} = M^{-1}\boldsymbol{p}$, $\dot{\boldsymbol{p}} = 0$) and V alone ("kick": $\dot{\boldsymbol{q}} = 0$, $\dot{\boldsymbol{p}} = -\nabla V$) can be solved **exactly**. Splitting methods are built from these exact sub-flows.
- **Symplecticity.** Let φ_t denote the exact flow, i.e. the map that sends an initial value $\boldsymbol{y}_0$ to the solution $\boldsymbol{y}(t)$.

Definition (symplectic map)
A differentiable map $\Psi : \mathbb{R}^{2m} \to \mathbb{R}^{2m}$ is *symplectic* if its Jacobian satisfies

$$\Psi'(\boldsymbol{y})^T J \Psi'(\boldsymbol{y}) = J \quad \text{for all } \boldsymbol{y}. \tag{6}$$

Theorem (Poincaré). The flow φ_t of a Hamiltonian system is symplectic for every t.

Proof. The Jacobian $\Psi(t) = \partial \varphi_t(\boldsymbol{y}_0)/\partial \boldsymbol{y}_0$ solves the variational equation $\dot{\Psi} = J\nabla^2 H(\boldsymbol{y}(t))\Psi$ with $\Psi(0) = I$. Using $J^T J = I$, $JJ = -I$ and the symmetry of the Hessian $\nabla^2 H$,

$$\frac{\mathrm{d}}{\mathrm{d}t}(\Psi^T J \Psi) = \Psi^T \nabla^2 H J^T J \Psi + \Psi^T J J \nabla^2 H \Psi = \Psi^T \nabla^2 H \Psi - \Psi^T \nabla^2 H \Psi = 0, \tag{7}$$

so $\Psi^T J \Psi = \Psi(0)^T J \Psi(0) = J$. □

Taking determinants of Equation 6 gives $\det \Psi' = 1$: symplectic maps preserve phase-space volume (Liouville's theorem). For $m = 1$ symplecticity is **equivalent** to area preservation; in higher dimensions it is a stronger property: the sum of oriented areas of projections onto the (q_k, p_k) planes is preserved [4]. Symplecticity is the property a numerical method must inherit to show the long-time behaviour of Hamiltonian dynamics (Section 3.5).

2.3 Conservation laws

The exact solution of Equation 2 has several invariants. They follow from symmetries of the Hamiltonian via Noether's theorem [8]: invariance under time translation gives conservation of energy, invariance under spatial translation conservation of momentum, and invariance under rotation conservation of angular momentum. Since these invariants are known exactly, their numerical violation is the most important accuracy diagnostic of an N-body code.

Energy. Along any solution,

$$\frac{\mathrm{d}H}{\mathrm{d}t} = \nabla H(\boldsymbol{y})^{T}\dot{\boldsymbol{y}} = \nabla H^{T} J \nabla H = 0, \tag{8}$$

because J is skew-symmetric. Hence $E = T + V$ is constant.

Linear momentum. The total momentum $\boldsymbol{P} = \sum_i m_i \boldsymbol{v}_i$ changes by the sum of all forces,

$$\dot{\boldsymbol{P}} = \sum_{i} \sum_{j \neq i} \boldsymbol{F}_{ij} = \sum_{i<j} \left(\boldsymbol{F}_{ij} + \boldsymbol{F}_{ji}\right) = \boldsymbol{0} \tag{9}$$

by Newton's third law. Consequently the centre of mass $\boldsymbol{R} = M^{-1}\sum_i m_i \boldsymbol{r}_i$ moves uniformly, $\boldsymbol{R}(t) = \boldsymbol{R}_0 + t\boldsymbol{P}/M$. All scenarios in this work are transformed to the barycentric frame $\boldsymbol{R} = \boldsymbol{P} = \boldsymbol{0}$.

Angular momentum. With $\boldsymbol{L} = \sum_i \boldsymbol{r}_i \times m_i \boldsymbol{v}_i$,

$$\dot{\boldsymbol{L}} = \sum_i \underbrace{\boldsymbol{v}_i \times m_i \boldsymbol{v}_i}_{=\boldsymbol{0}} + \sum_i \boldsymbol{r}_i \times \sum_{j\neq i} \boldsymbol{F}_{ij} = \sum_{i<j} \left(\boldsymbol{r}_i - \boldsymbol{r}_j\right) \times \boldsymbol{F}_{ij} = \boldsymbol{0}, \tag{10}$$

because $\boldsymbol{F}_{ij}$ is parallel to $\boldsymbol{r}_j - \boldsymbol{r}_i$. In two dimensions only the component $L_z = \sum_i m_i\left(x_i v_{y,i} - y_i v_{x,i}\right)$ exists.

Virial theorem. The moment of inertia $I = \sum_i m_i \, |\boldsymbol{r}_i|^2$ satisfies $\dot{I} = 2\sum_i m_i \boldsymbol{r}_i \cdot \boldsymbol{v}_i$ and

$$\frac{1}{2}\ddot{I} = \sum_i m_i \, |\boldsymbol{v}_i|^2 + \sum_i \boldsymbol{r}_i \cdot \boldsymbol{F}_i = 2T + \sum_{i<j} \left(\boldsymbol{r}_i - \boldsymbol{r}_j\right) \cdot \boldsymbol{F}_{ij} = 2T + V, \tag{11}$$

where we used $\left(\boldsymbol{r}_i - \boldsymbol{r}_j\right)\cdot \boldsymbol{F}_{ij} = -G m_i m_j / r_{ij}$. For a bound system $\dot{I}$ stays bounded, so the time average of $\ddot{I}$ vanishes and $2\langle T\rangle + \langle V\rangle = 0$. A system in virial equilibrium therefore has the **virial ratio** $Q = T/\,|V| = 1/2$. We use this to set up star clusters in equilibrium (Section 4.7).

2.4 The two-body problem

For $N = 2$ the problem can be solved exactly. The solution is our most important reference for measuring global errors.

Reduction. Introduce the centre of mass $\boldsymbol{R}$ and the relative coordinate $\boldsymbol{r} = \boldsymbol{r}_2 - \boldsymbol{r}_1$. Subtracting the two equations of motion gives

$$\ddot{\boldsymbol{r}} = -\frac{\mu}{r^{3}}\boldsymbol{r}, \quad \mu = G(m_1 + m_2), \tag{12}$$

and the bodies move as $\boldsymbol{r}_1 = \boldsymbol{R} - (m_2/M)\boldsymbol{r}$, $\boldsymbol{r}_2 = \boldsymbol{R} + (m_1/M)\boldsymbol{r}$.

Planarity and Kepler's second law. The specific angular momentum $\boldsymbol{h} = \boldsymbol{r} \times \dot{\boldsymbol{r}}$ is constant because the force is central. Hence the motion takes place in the plane perpendicular to $\boldsymbol{h}$, and the area swept out per unit time, $|\boldsymbol{h}|\,/2$, is constant.

Orbit equation. The Laplace–Runge–Lenz vector $\boldsymbol{A} = \dot{\boldsymbol{r}} \times \boldsymbol{h} - \mu \boldsymbol{r}/r$ is conserved, since

$$\frac{\mathrm{d}}{\mathrm{d}t}(\dot{\boldsymbol{r}} \times \boldsymbol{h}) = -\frac{\mu}{r^3}\boldsymbol{r} \times (\boldsymbol{r} \times \dot{\boldsymbol{r}}) = \mu\left(\frac{\dot{\boldsymbol{r}}}{r} - \frac{(\boldsymbol{r}\cdot\dot{\boldsymbol{r}})\boldsymbol{r}}{r^3}\right) = \mu \frac{\mathrm{d}}{\mathrm{d}t}\frac{\boldsymbol{r}}{r}. \tag{13}$$

Let f (the *true anomaly*) be the angle between $\boldsymbol{r}$ and $\boldsymbol{A} = \mu \boldsymbol{e}$. Using $\boldsymbol{r}\cdot(\dot{\boldsymbol{r}} \times \boldsymbol{h}) = (\boldsymbol{r} \times \dot{\boldsymbol{r}})\cdot\boldsymbol{h} = h^2$,

$$\boldsymbol{r}\cdot\boldsymbol{A} = h^2 - \mu r = \mu r e \cos f \quad\Longrightarrow\quad r(f) = \frac{h^2/\mu}{1 + e\cos f}, \tag{14}$$

a conic section with eccentricity $e = |\boldsymbol{e}|$ and focus at the origin (Kepler's first law). For $0 \le e < 1$ the orbit is an ellipse with semi-major axis $a = h^2/(\mu(1-e^2))$; its closest point to the focus (*periapsis*) lies in the direction of $\boldsymbol{e}$ at distance $r_p = a(1-e)$.

Energy, vis-viva and Kepler's third law. At periapsis $v_p = h/r_p$, so the specific orbital energy is

$$\varepsilon = \frac{v_p^2}{2} - \frac{\mu}{r_p} = \frac{\mu(1+e)}{2a(1-e)} - \frac{\mu}{a(1-e)} = -\frac{\mu}{2a}. \tag{15}$$

Since ε is conserved, we obtain the vis-viva equation $v^2 = \mu(2/r - 1/a)$ for all points of the orbit. Dividing the area $\pi a b$ of the ellipse, $b = a\sqrt{1-e^2}$, by the areal velocity $h/2$ gives Kepler's third law

$$P_{\mathrm{orb}} = \frac{2\pi a b}{h} = 2\pi\sqrt{\frac{a^3}{\mu}} = 2\pi\sqrt{\frac{a^3}{G(m_1+m_2)}}. \tag{16}$$

Kepler's equation. To obtain the position as a function of time, parametrise the ellipse by the *eccentric anomaly* E: $x = a(\cos E - e)$, $y = b\sin E$, with periapsis on the x-axis. The angular momentum becomes

$$h = x\dot{y} - y\dot{x} = ab\dot{E}(1 - e\cos E), \tag{17}$$

and with $h = \sqrt{\mu a(1-e^2)} = nab$, where $n = \sqrt{\mu/a^3}$ is the mean motion, we get $\dot{E}(1 - e\cos E) = n$. Integrating from periapsis at $t = 0$ yields Kepler's equation

$$E - e\sin E = nt =: \mathcal{M}. \tag{18}$$

Exact reference solution

Given t, solve Equation 18 for E with Newton's method, $E_{k+1} = E_k - (E_k - e\sin E_k - \mathcal{M})/(1 - e\cos E_k)$, which converges quadratically from $E_0 = \mathcal{M} + e\sin\mathcal{M}$ (or $E_0 = \pi$ for $e > 0.8$). Then

$$\boldsymbol{r}(t) = \begin{pmatrix} a(\cos E - e) \\ b\sin E \end{pmatrix}, \quad \dot{\boldsymbol{r}}(t) = \frac{n}{1 - e\cos E}\begin{pmatrix} -a\sin E \\ b\cos E \end{pmatrix}. \tag{19}$$

This closed-form solution is implemented in `nbody.analysis.kepler` and is used to measure global errors and convergence orders (Section 5.3).

2.5 Three and more bodies: non-integrability and chaos

For $N \geq 3$ the conserved quantities of Section 2.3 are not enough to reduce the problem to quadratures. Only special solutions are known in closed form, for example Lagrange's equilateral configurations or the *figure-eight* choreography of three equal masses, found numerically by Moore and proven to exist by Chenciner and Montgomery [9]. Generic configurations are chaotic. A famous example is Burrau's *Pythagorean problem* [10], in which three bodies with masses 3, 4, 5 start at rest at the vertices of a right triangle with side lengths 3, 4, 5. After a sequence of very close encounters the system breaks up into a binary and an escaping body [11].

Sensitivity to initial conditions. Small perturbations $\delta\boldsymbol{y}$ of a trajectory evolve according to the linearised equation $\delta\dot{\boldsymbol{y}} = J\nabla^2 H(\boldsymbol{y}(t))\delta\boldsymbol{y}$. In chaotic regions they grow exponentially on average,

$$|\delta\boldsymbol{y}(t)| \approx |\delta\boldsymbol{y}(0)|\, e^{\lambda t}, \quad \lambda = \lim_{t\to\infty}\frac{1}{t}\ln\frac{|\delta\boldsymbol{y}(t)|}{|\delta\boldsymbol{y}(0)|}, \tag{20}$$

where $\lambda > 0$ is the largest Lyapunov exponent. Numerical errors act like such perturbations. If a solution must be known to a tolerance Δ and the initial error is δ_0, the **predictability horizon** is

$$t_{\mathrm{pred}} \approx \frac{1}{\lambda}\ln\frac{\Delta}{\delta_0}. \tag{21}$$

Each additional digit of accuracy extends t_{pred} only by $\ln(10)/\lambda$. Beyond t_{pred}, comparing individual trajectories point by point is meaningless. A long chaotic integration can only be judged through invariants (energy, angular momentum) and statistical properties. This

is why conservation errors are the primary metric of Section 5, and pointwise errors are only used for integrable or stable problems.

2.6 Gravitational softening

The singularity of Equation 1 at $r_{ij} = 0$ causes arbitrarily large accelerations during close encounters. For *collisionless* systems such as star clusters or galaxies, where the bodies represent a smooth mass distribution, one replaces the point-mass potential by the Plummer potential [1], [12]:

$$V_\varepsilon(\boldsymbol{q}) = -\sum_{i<j} \frac{G m_i m_j}{\sqrt{r_{ij}^2 + \varepsilon^2}}. \tag{22}$$

Differentiating with $\partial r_{ij}^2 / \partial \boldsymbol{r}_i = -2\boldsymbol{r}_{ij}$ gives the softened acceleration

$$\boldsymbol{a}_i = -\frac{1}{m_i}\frac{\partial V_\varepsilon}{\partial \boldsymbol{r}_i} = G\sum_{j\neq i} m_j \frac{\boldsymbol{r}_{ij}}{\left(r_{ij}^2 + \varepsilon^2\right)^{3/2}}, \tag{23}$$

which is bounded by $\mathcal{O}(Gm/\varepsilon^2)$ and equals the Newtonian force for $r_{ij} \gg \varepsilon$. The choice of ε trades force bias at small separations against discreteness noise [13]. For planetary systems and few-body problems we use $\varepsilon = 0$.

Consistency of force and energy

The softened system is again Hamiltonian, with $H_\varepsilon = T + V_\varepsilon$. All results of Section 2.2 and Section 2.3 remain valid, but the conserved energy is $E_\varepsilon = T + V_\varepsilon$ and **not** $T + V$. Mixing the softened force Equation 23 with the Newtonian potential in the energy diagnostic produces an apparent energy error even for the exact solution, because for $r \gg \varepsilon$

$$V - V_\varepsilon \approx -\sum_{i<j} \frac{G m_i m_j \varepsilon^2}{2 r_{ij}^3} \tag{24}$$

varies along the orbit. The original course project used exactly this inconsistent diagnostic. Section 5.5 quantifies the effect.

2.7 Units and scaling

Newton's equations are invariant under the scaling $\boldsymbol{r} \to \alpha \boldsymbol{r}$, $t \to \beta t$, $m \to \gamma m$ if $\alpha^3 = \gamma \beta^2$. Results obtained in dimensionless units therefore transfer to any physical scale. We use two unit systems:

- **N-body units** with $G = 1$ and characteristic mass and length of order one, for test problems. The Kepler orbit with $a = 1$, $m_1 + m_2 \approx 1$ then has period $P_{\mathrm{orb}} \approx 2\pi$.
- **Astronomical units** (AU, year, solar mass). Applying Kepler's third law Equation 16 to the Earth ($a = 1\ \mathrm{AU}$, $P_{\mathrm{orb}} = 1\ \mathrm{yr}$, $m_\oplus \ll M_\odot$) gives

$$G = 4\pi^2 \,\mathrm{AU}^3 M_\odot^{-1} \mathrm{yr}^{-2}. \tag{25}$$

Choosing units in which the relevant quantities are of order one also keeps the floating-point arithmetic well conditioned. The original project had to rescale SI values by $1/2.5 \cdot 10^{11}$ for plotting.

3 Numerical Integration of Hamiltonian Systems

This chapter develops the integrators studied in this thesis in order of increasing sophistication. For each method we derive its order of accuracy, its stability on the linear test problem and whether it preserves the symplectic structure. These are the three properties that the experiments of Section 5 test. The presentation follows Hairer, Lubich and Wanner [4] and Hairer, Nørsett and Wanner [14].

3.1 One-step methods, order and cost

We write the equations of motion as a first-order system $\dot{\boldsymbol{y}} = \boldsymbol{f}(\boldsymbol{y})$ with $\boldsymbol{y} = (\boldsymbol{x}, \boldsymbol{v})$ and $\boldsymbol{f}(\boldsymbol{y}) = (\boldsymbol{v}, \boldsymbol{a}(\boldsymbol{x}))$. A *one-step method* is a map Φ_h that advances the numerical solution by one time step h, $\boldsymbol{y}_{n+1} = \Phi_h(\boldsymbol{y}_n)$. It approximates the exact flow φ_h.

Definition (order)
The *local error* of Φ_h is $\boldsymbol{\delta}(\boldsymbol{y}, h) = \Phi_h(\boldsymbol{y}) - \varphi_h(\boldsymbol{y})$. The method has *order* p if $\boldsymbol{\delta} = \mathcal{O}(h^{p+1})$ for all smooth $\boldsymbol{f}$.

From local to global error. Let $\boldsymbol{e}_n = \boldsymbol{y}_n - \boldsymbol{y}(t_n)$ and assume Φ_h is Lipschitz with constant $1 + hL$. Then $|\boldsymbol{e}_{n+1}| \le (1 + hL)|\boldsymbol{e}_n| + Ch^{p+1}$. Summing this geometric recursion ("Lady Windermere's fan" [14]) gives

$$|\boldsymbol{e}_n| \le Ch^{p+1} \sum_{k=0}^{n-1} (1 + hL)^k \le \frac{Ch^p}{L}\left(e^{Lt_n} - 1\right). \tag{26}$$

A method of order p therefore has global error $\mathcal{O}(h^p)$ on bounded intervals. The constant may grow exponentially with t, and this bound says nothing about which errors accumulate. Answering that question requires the geometric analysis of Section 3.5.

Cost. In all methods considered here the evaluation of the accelerations $\boldsymbol{a}(\boldsymbol{x})$, which costs $\mathcal{O}(N^2)$, dominates the cost. We therefore measure cost in *force evaluations* and compare methods at equal cost rather than at equal step size. A method with s evaluations per step must take steps that are s times larger to be as cheap as a method with one evaluation per step.

Linear test problem. The harmonic oscillator $\ddot{x} = -\omega^2 x$ is the model problem for oscillatory motion. Every method below is linear on it, $\boldsymbol{y}_{n+1} = M(h\omega)\boldsymbol{y}_n$ with $\boldsymbol{y} = (x, v)$. The method is **stable** if the powers M^n stay bounded. For the 2×2 matrices considered here, which have distinct eigenvalues away from the stability boundary, this is equivalent to a spectral radius $\rho(M) \le 1$. The exact flow is a rotation in the $(\omega x, v)$ plane with $|\det| = 1$ and eigenvalues $e^{\pm i\omega h}$ on the unit circle. Its energy $E = (v^2 + \omega^2 x^2)/2$ is conserved.

3.2 Explicit Euler method

The simplest method follows from the Taylor expansion $\boldsymbol{y}(t+h) = \boldsymbol{y}(t) + h\boldsymbol{f}(\boldsymbol{y}(t)) + \mathcal{O}(h^2)$:

$$\boldsymbol{x}_{n+1} = \boldsymbol{x}_n + h\boldsymbol{v}_n, \quad \boldsymbol{v}_{n+1} = \boldsymbol{v}_n + h\boldsymbol{a}(\boldsymbol{x}_n). \tag{27}$$

Its local error is $\frac{h^2}{2}\ddot{\boldsymbol{y}} + \mathcal{O}(h^3)$, so the method has order $p = 1$ and needs one force evaluation per step.

On the harmonic oscillator,

$$M_{\mathrm{E}} = \begin{pmatrix} 1 & h \\ -h\omega^2 & 1 \end{pmatrix}, \quad \det M_{\mathrm{E}} = 1 + h^2\omega^2, \quad \rho(M_{\mathrm{E}}) = \sqrt{1 + h^2\omega^2} > 1. \tag{28}$$

The method is **unstable for every step size**. The energy grows by the factor $1 + h^2\omega^2$ in every step, so the numerical orbit spirals outwards (Figure 5.1). For the Kepler problem the same mechanism lets the energy increase steadily until the planet escapes.

3.3 Symplectic Euler method

Exchanging the order of the two updates in Equation 27, so that the position is advanced with the *new* velocity, gives the symplectic (semi-implicit) Euler method

$$\boldsymbol{v}_{n+1} = \boldsymbol{v}_n + h\boldsymbol{a}(\boldsymbol{x}_n), \quad \boldsymbol{x}_{n+1} = \boldsymbol{x}_n + h\boldsymbol{v}_{n+1}. \tag{29}$$

It is still explicit, still of order one and still needs one force evaluation per step, yet its long-time behaviour is completely different.

Symplecticity. The first update of Equation 29 is the exact time-h flow φ_h^V of the sub-Hamiltonian $V(\boldsymbol{q})$ (a "kick": positions fixed, momenta change by $-h\nabla V$). The second is the exact flow φ_h^T of $T(\boldsymbol{p})$ (a "drift"). Hence

$$\Phi_h^{\mathrm{SE}} = \varphi_h^T \circ \varphi_h^V \tag{30}$$

is a composition of two exact Hamiltonian flows. Both are symplectic by Poincaré's theorem (Section 2.2), and since $(AB)^T J (AB) = B^T (A^T J A) B = B^T J B = J$, so is their composition.

Harmonic oscillator. Here

$$M_{\mathrm{SE}} = \begin{pmatrix} 1 - h^2\omega^2 & h \\ -h\omega^2 & 1 \end{pmatrix}, \quad \det M_{\mathrm{SE}} = 1, \quad \operatorname{tr} M_{\mathrm{SE}} = 2 - h^2\omega^2. \tag{31}$$

For $h\omega < 2$ the eigenvalues are complex conjugates of modulus one, so the method is stable and neither damps nor amplifies. A direct calculation shows that the quadratic form

$$\tilde{E}_{\mathrm{SE}} = \frac{1}{2}(v^2 + \omega^2 x^2 - h\omega^2 x v) \tag{32}$$

is **exactly** conserved: with $x' = (1 - h^2\omega^2)x + hv$ and $v' = v - h\omega^2 x$ all terms of order h and higher cancel in $v'^2 + \omega^2 x'^2 - h\omega^2 x' v'$. The true energy $E = \tilde{E}_{\mathrm{SE}} + \frac{h\omega^2}{2} x v$ therefore oscillates with amplitude $\mathcal{O}(h)$ but never drifts. The numerical orbit is a slightly tilted ellipse rather than an outward spiral (Figure 5.1).

> **Remark: symplectic Euler is conjugate to leapfrog**
> Symplectic Euler and the leapfrog method of the next section generate the same sequence of positions. Symplectic Euler's velocity $\boldsymbol{v}_n$ approximates the leapfrog half-step velocity $\boldsymbol{v}_{n-1/2}$, which differs from $\boldsymbol{v}(t_n)$ by $\frac{h}{2}\boldsymbol{a}(\boldsymbol{x}_n)$. Its global error is therefore $\mathcal{O}(h)$ in general. At special instants this $\mathcal{O}(h)$ term can cancel, and the method then *appears* to be of second order. For a Kepler orbit started at periapsis this happens after every full period, because the offset is perpendicular to the velocity there. For this reason Section 5.3 measures the **maximum** error over the whole time interval.

3.4 Störmer–Verlet / leapfrog

Derivation by splitting. The exact sub-flows φ^T and φ^V can be combined symmetrically (Strang splitting [15]):

$$\Phi_h^{\mathrm{LF}} = \varphi^V_{h/2} \circ \varphi^T_h \circ \varphi^V_{h/2}, \tag{33}$$

which written out is the *velocity Verlet* or *kick–drift–kick leapfrog* method [16]:

$$\begin{aligned} \boldsymbol{v}_{n+1/2} &= \boldsymbol{v}_n + \frac{h}{2}\boldsymbol{a}(\boldsymbol{x}_n), \\ \boldsymbol{x}_{n+1} &= \boldsymbol{x}_n + h\boldsymbol{v}_{n+1/2}, \\ \boldsymbol{v}_{n+1} &= \boldsymbol{v}_{n+1/2} + \frac{h}{2}\boldsymbol{a}(\boldsymbol{x}_{n+1}). \end{aligned} \tag{34}$$

Eliminating the velocities gives Störmer's two-step form $\boldsymbol{x}_{n+1} - 2\boldsymbol{x}_n + \boldsymbol{x}_{n-1} = h^2\boldsymbol{a}(\boldsymbol{x}_n)$, the central-difference discretisation of $\ddot{\boldsymbol{x}} = \boldsymbol{a}$.

Cost. The acceleration $\boldsymbol{a}(\boldsymbol{x}_{n+1})$ computed at the end of a step is the one needed at the beginning of the next ("first same as last", FSAL). If it is cached, every step after the first costs **one** force evaluation, the same as the Euler methods. The original course implementation recomputed it and used two evaluations per step.

Properties.

- *Symplectic*, as a composition of exact Hamiltonian flows.
- *Symmetric (time-reversible)*: $\Phi_{-h} \circ \Phi_h = \mathrm{id}$. In $\varphi^V_{-h/2}\varphi^T_{-h}\varphi^V_{-h/2}\varphi^V_{h/2}\varphi^T_h\varphi^V_{h/2}$ the inner kicks cancel, then the drifts, then the outer kicks. Integrating forward and then backward with $-h$ returns exactly to the initial state, up to round-off.
- *Order two.* The method is consistent, and a symmetric method has even order: if $\Phi_h = \varphi_h + Ch^{p+1} + \mathcal{O}(h^{p+2})$, then its adjoint Φ_{-h}^{-1} has leading error $(-1)^p Ch^{p+1}$ [4]. Symmetry, $\Phi_h = \Phi_{-h}^{-1}$, forces $C = (-1)^p C$, which for $C \neq 0$ requires p even.

Harmonic oscillator. Multiplying the three linear maps gives

$$M_{\mathrm{LF}} = \begin{pmatrix} 1 - \frac{h^2\omega^2}{2} & h \\ -h\omega^2\left(1 - \frac{h^2\omega^2}{4}\right) & 1 - \frac{h^2\omega^2}{2} \end{pmatrix}, \quad \det M_{\mathrm{LF}} = 1, \quad \operatorname{tr} M_{\mathrm{LF}} = 2 - h^2\omega^2. \tag{35}$$

The eigenvalues are $e^{\pm i\theta}$ with $\cos\theta = 1 - h^2\omega^2/2$, so the method is stable if and only if $h\omega < 2$. Writing $\theta = \tilde{\omega}h$ defines the **numerical frequency**

$$\tilde{\omega} = \frac{2}{h}\arcsin\frac{h\omega}{2} = \omega\left(1 + \frac{h^2\omega^2}{24} + \mathcal{O}(h^4)\right). \tag{36}$$

The amplitude is exact, but the phase advances slightly too fast, and the phase error grows **linearly** in time. As for symplectic Euler, a modified energy is conserved exactly:

$$\tilde{E}_{\mathrm{LF}} = \frac{1}{2}\left(v^2 + \omega^2\left(1 - \frac{h^2\omega^2}{4}\right)x^2\right). \tag{37}$$

This can be checked by direct substitution: with $c = 1 - h^2\omega^2/2$ and $k = 1 - h^2\omega^2/4$ one has $c^2 + kh^2\omega^2 = 1$, which is exactly the condition that $v'^2 + k\omega^2 x'^2 = v^2 + k\omega^2 x^2$.

3.5 Backward error analysis and long-time behaviour

The modified energies Equation 32 and Equation 37 are special cases of a general theorem that explains the excellent long-time behaviour of symplectic integrators.

Modified equations. For a method of order p one can construct a *modified differential equation*

$$\dot{\tilde{\boldsymbol{y}}} = \boldsymbol{f}(\tilde{\boldsymbol{y}}) + h^p \boldsymbol{f}_{p+1}(\tilde{\boldsymbol{y}}) + h^{p+1}\boldsymbol{f}_{p+2}(\tilde{\boldsymbol{y}}) + \dots \tag{38}$$

whose exact solution reproduces the numerical solution, $\tilde{\boldsymbol{y}}(nh) = \boldsymbol{y}_n$, as a formal series. This is backward error analysis: the numerical solution is the exact solution of a nearby problem.

Theorem (modified Hamiltonian, [4, Ch. IX], [17])
If a method of order p is symplectic and applied to $\dot{\boldsymbol{y}} = J\nabla H(\boldsymbol{y})$, its modified equation is again Hamiltonian, $\dot{\tilde{\boldsymbol{y}}} = J\nabla\tilde{H}(\tilde{\boldsymbol{y}})$, with

$$\tilde{H} = H + h^p H_{p+1} + h^{p+1}H_{p+2} + \dots \tag{39}$$

After optimal truncation of the series, the numerical solution conserves $\tilde{H}$ up to an error that is exponentially small in $1/h$. Consequently

$$|H(\boldsymbol{y}_n) - H(\boldsymbol{y}_0)| = \mathcal{O}(h^p) \quad \text{for exponentially long times } nh \le e^{c/h}. \tag{40}$$

A non-symplectic method has a modified equation that is not Hamiltonian. Generically the energy then drifts linearly, $H(\boldsymbol{y}_n) - H(\boldsymbol{y}_0) \approx ch^p t$. Depending on the sign of c, the orbit gains energy (explicit Euler) or loses it (RK4 on the harmonic oscillator, Section 3.6).

Error growth for integrable systems. The Kepler problem shows how this difference affects the global error [4, Ch. X], [18]. The mean motion depends on the energy, $n(E) = (-2E)^{3/2}/\mu$, which follows from Equation 15 and Equation 16. The error in the orbital phase φ accumulates as

$$\delta\varphi(t) = \int_0^t \delta n \, \mathrm{d}s, \quad \delta n \approx n'(E)\delta E(s) + \delta n_{\mathrm{num}}, \tag{41}$$

where $\delta n_{\mathrm{num}} = \mathcal{O}(h^p)$ is the frequency error of the method, as in Equation 36.

- For a **symplectic** method $\delta E = \mathcal{O}(h^p)$ stays bounded, so $\delta\varphi \approx c_1 h^p t$: the global error grows **linearly** in time.
- For a **non-symplectic** method $\delta E \approx ch^p t$, so $\delta\varphi \approx \frac{1}{2}n'(E)ch^p t^2$: the global error grows **quadratically** in time.

Since the along-track position error is $a\delta\varphi$, after long times the symplectic method wins regardless of the error constants. Section 5.4 confirms both growth laws.

3.6 Runge–Kutta methods

An explicit s-stage Runge–Kutta method is defined by its Butcher tableau $\left(a_{ij}\right)_{j<i}$, b_i, $c_i = \sum_j a_{ij}$ [19]:

$$\boldsymbol{k}_i = \boldsymbol{f}\left(\boldsymbol{y}_n + h\sum_{j<i} a_{ij}\boldsymbol{k}_j\right), \quad i = 1, \ldots, s, \quad \boldsymbol{y}_{n+1} = \boldsymbol{y}_n + h\sum_{i=1}^{s} b_i \boldsymbol{k}_i. \tag{42}$$

Expanding the exact and numerical solutions in Taylor series and comparing coefficients gives the order conditions. For order four these are the eight equations

$$\begin{gathered}\sum b_i = 1, \quad \sum b_i c_i = \frac{1}{2}, \quad \sum b_i c_i^2 = \frac{1}{3}, \quad \sum b_i a_{ij} c_j = \frac{1}{6}, \\ \sum b_i c_i^3 = \frac{1}{4}, \quad \sum b_i c_i a_{ij} c_j = \frac{1}{8}, \quad \sum b_i a_{ij} c_j^2 = \frac{1}{12}, \quad \sum b_i a_{ij} a_{jk} c_k = \frac{1}{24}.\end{gathered} \tag{43}$$

Two classical solutions are studied here:

Heun's method (RK2), $p = 2$

$$\begin{array}{c|cc} 0 & & \\ 1 & 1 & \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array}$$

Classical RK4, $p = 4$

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}$$

For $\dot{\boldsymbol{x}} = \boldsymbol{v}$, $\dot{\boldsymbol{v}} = \boldsymbol{a}(\boldsymbol{x})$ every stage needs one force evaluation, so Heun costs two and RK4 four evaluations per step.

Stability function. Applied to $\dot{y} = \lambda y$, Equation 42 gives $y_{n+1} = R(h\lambda) y_n$ with $R(z) = 1 + z + z^2/2 + \ldots + z^p/p!$ for the methods above. The harmonic oscillator has $\lambda = \pm i\omega$, so stability on the imaginary axis is what matters. With $z = iy$, $y = h\omega$:

$$\begin{aligned} |R_{\text{Euler}}(iy)|^2 &= 1 + y^2, \\ |R_{\text{Heun}}(iy)|^2 &= \left(1 - y^2/2\right)^2 + y^2 = 1 + y^4/4, \\ |R_{\text{RK4}}(iy)|^2 &= \left(1 - y^2/2 + y^4/24\right)^2 + \left(y - y^3/6\right)^2 = 1 - \frac{y^6}{72} + \frac{y^8}{576}. \end{aligned} \tag{44}$$

Explicit Euler and Heun amplify for every $h > 0$, so their energy grows. RK4 is stable for $y^8/576 \le y^6/72$, i.e. $h\omega \le 2\sqrt{2} \approx 2.83$, but **dissipates** energy by about $(h\omega)^6/72$ per step. RK4 has the largest stability interval of all methods considered here, but none of them is energy-preserving in the long run.

No explicit Runge–Kutta method is symplectic. A Runge–Kutta method is symplectic if and only if $b_i a_{ij} + b_j a_{ji} = b_i b_j$ for all i, j [4, Thm. VI.4.3]. For $i = j$ this requires $2b_i a_{ii} = b_i^2$. An explicit method has $a_{ii} = 0$, so all $b_i = 0$, which contradicts $\sum b_i = 1$. Symplectic methods must therefore be built differently: by splitting (Section 3.4) or by composition (Section 3.7).

3.7 Composition methods

High-order symplectic methods are obtained by composing a basic symplectic method with suitably chosen step sizes [20], [21], [22]. Let Φ_h be symmetric and of even order p, with local error $\Phi_h = \varphi_h + h^{p+1}\boldsymbol{C} + \mathcal{O}(h^{p+2})$, and consider

$$\Psi_h = \Phi_{\gamma_s h} \circ \cdots \circ \Phi_{\gamma_1 h}. \tag{45}$$

Since flows compose additively in time and the leading error terms add up at lowest order,

$$\Psi_h = \varphi_{(\gamma_1+\ldots+\gamma_s)h} + \left(\gamma_1^{p+1} + \ldots + \gamma_s^{p+1}\right) h^{p+1} \boldsymbol{C} + \mathcal{O}(h^{p+2}). \tag{46}$$

Ψ_h thus has order at least $p+1$ if

$$\sum_k \gamma_k = 1 \quad \text{(consistency)}, \qquad \sum_k \gamma_k^{p+1} = 0 \quad \text{(cancellation)}. \tag{47}$$

If the weights are also palindromic ($\gamma_k = \gamma_{s+1-k}$), Ψ_h is symmetric, so its order is even and hence at least $p+2$.

Triple jump. With $s = 3$ and $\gamma_1 = \gamma_3$, Equation 47 becomes $2\gamma_1 + \gamma_2 = 1$ and $2\gamma_1^{p+1} + \gamma_2^{p+1} = 0$. Since $p+1$ is odd, the second condition gives $\gamma_2 = -2^{1/(p+1)}\gamma_1$, and therefore

$$\gamma_1 = \gamma_3 = \frac{1}{2 - 2^{1/(p+1)}}, \quad \gamma_2 = -\frac{2^{1/(p+1)}}{2 - 2^{1/(p+1)}}. \tag{48}$$

- Applied to leapfrog ($p = 2$), Equation 48 gives $\gamma_1 \approx 1.3512$, $\gamma_2 \approx -1.7024$. This is **Yoshida's fourth-order method** [20] (equivalently Forest and Ruth [23]). With FSAL it costs three force evaluations per step.
- Applied once more to Yoshida-4 ($p = 4$), the recursion yields a sixth-order method with $3^2 = 9$ leapfrog stages (9 evaluations).
- Yoshida also solved the order conditions for seven symmetric stages directly. This **sixth-order "solution A"** needs only 7 evaluations per step (coefficients in Section B).

All composition methods are symplectic (compositions of symplectic maps) and time-reversible (palindromic). Their price is the **negative** intermediate step $\gamma_2 h$, which makes the error constants larger and the stability interval smaller. On the harmonic oscillator Yoshida-4 is stable only for $h\omega < 1.575$, about 0.53 per force evaluation, compared with 2 for leapfrog (Section 5.2). Increasing the order is therefore only worthwhile when high accuracy is required.

3.8 Adaptive step-size control

Fixed step sizes are inefficient when the dynamics have very different time scales, e.g. a highly eccentric orbit or a close encounter. **Embedded Runge–Kutta pairs** estimate the local error at no extra cost. The Dormand–Prince pair DOPRI5(4) [24] uses seven stages to compute a fifth-order solution $\boldsymbol{y}_{n+1}$ and an embedded fourth-order solution $\hat{\boldsymbol{y}}_{n+1}$. Their difference estimates the local error of the fourth-order solution, which is $\mathcal{O}(h^5)$.

Error norm. Each component is scaled with mixed absolute and relative tolerances,

$$\mathrm{err} = \sqrt{\frac{1}{n}\sum_{k=1}^{n}\left(\frac{y_{n+1,k} - \hat{y}_{n+1,k}}{\mathrm{atol} + \mathrm{rtol}\cdot\max\bigl(|y_{n,k}|, |y_{n+1,k}|\bigr)}\right)^2}. \tag{49}$$

Step-size controller. A step is accepted if $\mathrm{err} \le 1$. Since $\mathrm{err} \approx C h^5$, the step size that would give $\mathrm{err} = 1$ is $h \cdot \mathrm{err}^{-1/5}$. With a safety factor $s = 0.9$ and limits on the change,

$$h_{\mathrm{new}} = h \cdot \min\bigl(f_{\max}, \max\bigl(f_{\min}, s \cdot \mathrm{err}^{-1/5}\bigr)\bigr), \quad f_{\min} = 0.2, \quad f_{\max} = 5. \tag{50}$$

After a rejected step the step is repeated with h_{new}, and no increase is allowed directly after a rejection [14]. The seventh stage is evaluated at the new solution (FSAL), so an accepted step costs six force evaluations.

Adaptivity versus geometry. Adaptivity is essential for close encounters (Section 5.7), but it is not free. Even a symplectic method loses its long-time properties if the step size is changed according to the current state, because the modified Hamiltonian depends on h [18]. DOPRI5 is in addition neither symplectic nor symmetric. Structure-preserving adaptivity, via time transformations or regularisation [1], [4, Ch. VIII], is discussed in Section 6.

3.9 Summary of the methods

Table 3.1 summarises the theoretical properties of all integrators. The stability limits were obtained numerically from the spectral radius of $M(h\omega)$ (Section 5.2) and agree with the analytic values 0, 2, 2 and $2\sqrt{2}$ derived above.

Table 3.1: Integrators studied in this thesis, in order of increasing sophistication. "Stability" is the largest $h\omega$ for which the method is stable on the harmonic oscillator (0: unstable for every $h > 0$).

Method	Order p	Symplectic	Symmetric	Evals/step	Stability $h\omega$
Explicit Euler	1	no	no	1	0
Symplectic Euler	1	yes	no	1	2.00
Heun (RK2)	2	no	no	2	0
Leapfrog / Verlet	2	yes	yes	1	2.00
Runge–Kutta 4	4	no	no	4	2.83
Yoshida 4	4	yes	yes	3	1.58
Yoshida 6 (solution A)	6	yes	yes	7	–
Yoshida 6 (triple jump)	6	yes	yes	9	–
Dormand–Prince 5(4)	5	no	no	6 (adaptive)	–

4 Implementation

This chapter describes the software that implements the methods of Section 3. The code is a Python package, `nbody`, built on NumPy [25], with Matplotlib [26] for plots and pygame for the interactive viewer. SciPy [27] is used only to compute independent reference solutions. The design follows four goals:

1. **Correctness.** Every theoretical property claimed in Section 3 (order, symplecticity, reversibility, cost) is checked by an automated test.
2. **Extensibility.** New integrators, force laws and scenarios can be added without modifying existing code.
3. **Efficiency.** All numerical kernels operate on whole arrays; no Python loop runs over bodies or pairs.
4. **Reproducibility.** Every figure and table in Section 5 is regenerated by a single command and read directly by this document.

4.1 Repository structure

The repository contains the code and this thesis side by side:

```
nbody-simulation/
├── src/nbody/              Python package
│   ├── core/               State, force models, force kernels
│   ├── integrators/        Euler, Verlet, Runge-Kutta, composition, adaptive
│   ├── analysis/           conserved quantities, metrics, exact Kepler solution
│   ├── scenarios/          builtin initial conditions, JSON scenario format
│   ├── viz/                plot style, static plots, real-time viewer + overlays
│   ├── simulation.py       simulation driver and trajectory container
│   └── cli.py              command-line interface (python -m nbody)
├── scenarios/              user-editable JSON scenarios
├── experiments/            evaluation experiments e01-e09, run_all.py
├── tests/                  pytest suite (100 tests)
├── scripts/                helper scripts (font download)
└── thesis/                 this document (Typst)
    ├── chapters/, lib/     sources
    ├── figures/generated/  figures written by the experiments
    └── data/               CSV/JSON results read by the thesis
```

4.2 Architecture

The package has a strict layered structure (Figure 4.1). Each layer depends only on the layers below it. The integrators know nothing about gravity: they see a `ForceModel` that returns accelerations. The simulation driver knows nothing about specific integrators: it sees the abstract `Integrator` interface. Visualisation and analysis only consume trajectories.

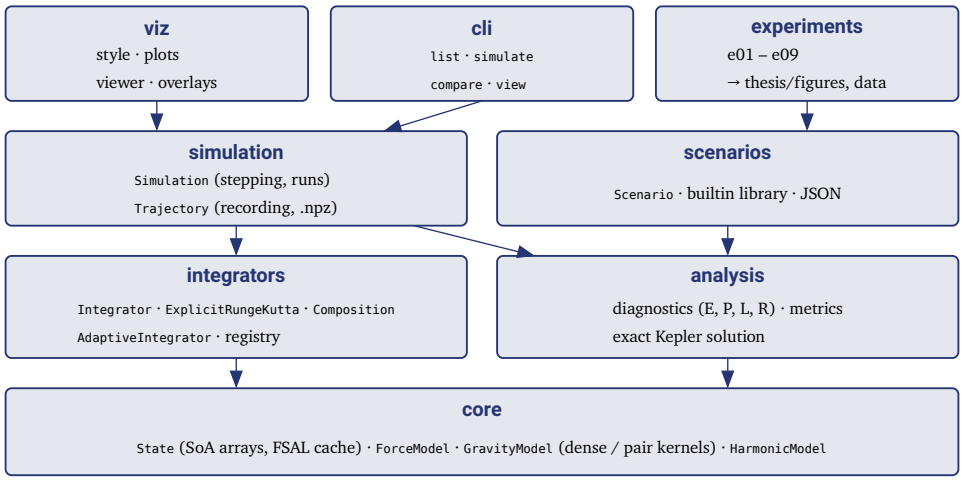


Figure 4.1: Layered architecture of the `nbody` package. Arrows point from a layer to the layers it uses.

4.3 Data model

Structure of arrays. The original implementation stored one `Body` object per particle and looped over them in Python. The new `State` class stores all positions in one array of shape (N, d) and all velocities in another. Every update is then a single vectorised NumPy operation, e.g. `state.x += dt * state.v` for a drift. The class also carries the current time and an optional cached acceleration `a`, which implements the FSAL optimisation of Section 3.4. Any code that changes `x` invalidates the cache.

Force models. A `ForceModel` represents a separable Hamiltonian $H = T(\boldsymbol{v}) + V(\boldsymbol{x})$. It provides `acceleration(x)` and `potential_energy(x)`, and it counts every force evaluation, which is the hardware-independent cost measure used in Section 5. `GravityModel` implements softened Newtonian gravity. `HarmonicModel` implements the linear test problem and demonstrates that all integrators are independent of the force law.

4.4 Vectorised force evaluation

Dense kernel. For N bodies, form the separation tensor $\Delta_{ijk} = x_{jk} - x_{ik}$ of shape (N, N, d) by broadcasting, then the matrix of weights

$$W_{ij} = \begin{cases} m_j / \left(|\Delta_{ij}|^2 + \varepsilon^2\right)^{3/2} & i \neq j \\ 0 & i = j \end{cases} \tag{51}$$

and finally contract, $a_{ik} = G \sum_j W_{ij} \Delta_{ijk}$. This is exactly Equation 23. The implementation sets the diagonal of r^2 to one before the division and zeroes the diagonal of W afterwards, so the self-interaction never produces $0/0$:

```
dx = x[np.newaxis, :, :] - xi[:, np.newaxis, :]   # (B, N, d) separations
r2 = np.einsum("ijk,ijk->ij", dx, dx) + softening**2
r2[rows, rows + offset] = 1.0                     # avoid 0/0 on the diagonal
w = masses[np.newaxis, :] / (r2 * np.sqrt(r2))
w[rows, rows + offset] = 0.0                      # no self-interaction
acc = np.einsum("ij,ijk->ik", w, dx)              # sum over j
```

The temporary tensor needs $\mathcal{O}(N^2 d)$ memory. For large N the rows are therefore processed in blocks of B bodies (`xi` above) so that at most a few million elements are allocated at a time. The result is independent of the block size, and a test verifies this.

Pair-list kernel. For small N the cost is dominated by the overhead of the NumPy calls, not by arithmetic. A second kernel visits each of the $P = N(N-1)/2$ unordered pairs once and computes the pair vectors $\boldsymbol{f}_p = \boldsymbol{r}_{ij} / \left(r_{ij}^2 + \varepsilon^2\right)^{3/2}$. It then distributes them with a precomputed matrix $S \in \mathbb{R}^{N \times P}$, where $S_{ip} = m_j$ and $S_{jp} = -m_i$:

$$\boldsymbol{a} = G S \boldsymbol{f}. \tag{52}$$

Newton's third law is built into S. Because S has NP entries, the model uses this kernel automatically only for $N \leq 24$. Both kernels agree to round-off, as the tests verify.

Energy. The potential energy uses the same softening as the force, i.e. it evaluates Equation 22. This makes the energy diagnostic consistent with the dynamics (Section 2.6). It also accepts a whole trajectory of shape (T, N, d) and is evaluated in time chunks to bound memory.

4.5 Integrators

Interface. Every integrator implements a single method that advances a state in place and declares its theoretical properties as attributes:

```
class Integrator(ABC):
    name: str; label: str
    order: int; symplectic: bool; time_reversible: bool; evals_per_step: int
    adaptive: bool = False

    @abstractmethod
    def step(self, state: State, dt: float, model: ForceModel) -> None: ...
```

The attributes are not just documentation. The test suite checks each of them against the behaviour of the implementation: it measures the convergence order, the reversibility error, the determinant of the one-step map and the number of force evaluations. As an example, the complete leapfrog step with FSAL caching is

```
def step(self, state, dt, model):
    half = 0.5 * dt
    state.v += half * cached_acceleration(state, model)  # kick
    state.x += dt * state.v                              # drift
    a_new = model.acceleration(state.x)
    state.v += half * a_new                              # kick
    state.a = a_new                                      # reuse in the next step
    state.t += dt
```

Generic building blocks. Two classes make whole families of methods available with a few lines each:

- `ExplicitRungeKutta` implements Equation 42 for an arbitrary Butcher tableau. At class creation (`__init_subclass__`) it checks that the tableau is explicit and consistent ($\sum_j a_{ij} = c_i$). Heun and RK4 are defined only by their tableaus.
- `Composition` implements $\Phi_{\gamma_s h} \circ \cdots \circ \Phi_{\gamma_1 h}$ for any base method. It checks the consistency condition $\sum \gamma_k = 1$, sets `symplectic` from the base method, and derives `time_reversible` from the base method together with the palindromic structure of the weights. Yoshida-4, Yoshida-6 (triple jump) and Yoshida-6A are composition objects with different weights.

Adaptive methods derive from `AdaptiveIntegrator`. Their `step` may reject trial steps internally and returns the step actually taken together with a proposal for the next step.

Registry. Integrators are created by name through a registry (`create_integrator("yoshida4")`). The registry also stores aliases such as `"verlet"` and `"rk45"`, and its insertion order defines the "increasing sophistication" ordering used in tables and plots. Adding a method requires no change to existing code. For example, Suzuki's fourth-order five-stage composition [21] can be added as follows:

```
from nbody.integrators import Composition, VelocityVerlet, register

s = 1 / (4 - 4 ** (1 / 3))
register("suzuki4", lambda: Composition(VelocityVerlet(), [s, s, 1 - 4 * s, s,
s],
                                        order=4, name="suzuki4", label="Suzuki 4"))
```

4.6 Simulation driver and trajectories

`Simulation` couples a force model, a state and an integrator. It can be advanced step by step, which the viewer uses, or run to a final time while recording a `Trajectory`. For fixed-step methods, `run(t_end, dt)` takes $n = \lceil (t_{\mathrm{end}} - t_0)/\ \mathrm{dt} \rceil$ steps of size $(t_{\mathrm{end}} - t_0)/n \le \mathrm{dt}$. This lands exactly on t_{end}, and computing $t_k = t_0 + kh$ avoids accumulating round-off in the time variable. For adaptive methods the last step is clipped. Every `record_every`-th state and always the final state are stored in preallocated arrays.

A `Trajectory` holds the time series t, $\boldsymbol{x}$ and $\boldsymbol{v}$, the masses, the model parameters and metadata: integrator, number of steps, force evaluations, rejected steps, wall-clock time, and for adaptive runs all accepted step sizes. It computes conserved quantities over the whole series in vectorised form, using the force model that generated it. It is stored as a compressed `.npz` file. The original project used `pickle`, which executes arbitrary code on loading.

4.7 Scenarios

Initial conditions are `Scenario` objects with masses, positions, velocities, G, ε, names, colours, units and recommended run parameters. They come from two sources:

- **Builtin scenarios**, generated by parameterised functions: the Kepler orbit with given a, e and mass ratio (Section 2.4); the figure-eight [9]; the Pythagorean problem [10]; the Solar System; a binary star with a circumbinary planet; and a Gaussian star cluster scaled to virial ratio $Q = 1/2$ (Section 2.3).
- **JSON files** in a documented format (Section A). The format is backwards compatible with the scenario files of the original project.

Solar System initial conditions. The planets are initialised from J2000 mean orbital elements [28]: semi-major axis a, eccentricity e and longitude of perihelion ϖ, with masses from the IAU mass ratios. In this simplified coplanar model every planet starts at its perihelion,

$$\boldsymbol{r}_p = a(1-e)\begin{pmatrix}\cos\varpi\\ \sin\varpi\end{pmatrix}, \quad \boldsymbol{v}_p = \sqrt{\frac{G(M_\odot + m)(1+e)}{a(1-e)}}\begin{pmatrix}-\sin\varpi\\ \cos\varpi\end{pmatrix}, \tag{53}$$

where the speed follows from the vis-viva equation at $r = a(1-e)$. Finally the whole system is shifted to the barycentric frame by subtracting $\boldsymbol{R}$ and $\boldsymbol{P}/M$. Without this shift the original implementation let the Sun drift steadily, because the planets' momenta were not balanced by the Sun.

4.8 Interactive viewer and trajectory overlays

The viewer (`python -m nbody view <scenario>`) renders the simulation in real time with pygame. Its camera can follow the centre of mass, and it supports zooming and panning. The integrator can be switched at runtime with the number keys. A head-up display shows the integrator, the step size, the number of force evaluations and the **live** conservation errors δE and $|\delta \boldsymbol{L}|$ / $|\boldsymbol{L}_0|$, so the difference between symplectic and non-symplectic methods can be watched as it develops.

Two overlays visualise trajectories (Figure 4.2):

- **Trails (past).** A ring buffer (`TrailBuffer`) of fixed capacity stores the most recent positions of all bodies in one preallocated array. Pushing a frame costs $\mathcal{O}(N)$ with no allocation. The trail is drawn as a polyline whose brightness fades with age.
- **Forecast (future).** The `TrajectoryPredictor` clones the running simulation, including the cached acceleration, and integrates the clone over a configurable horizon with the same integrator. The forecast is drawn as dotted lines ending in a small ring at the predicted position. Because the clone is independent, the running simulation is not

affected; a test checks this and checks that the forecast coincides with the actual future trajectory. The forecast is recomputed every ten frames. For planetary systems it shows the orbits ahead of time, and for chaotic systems it shows how quickly predictions lose validity.

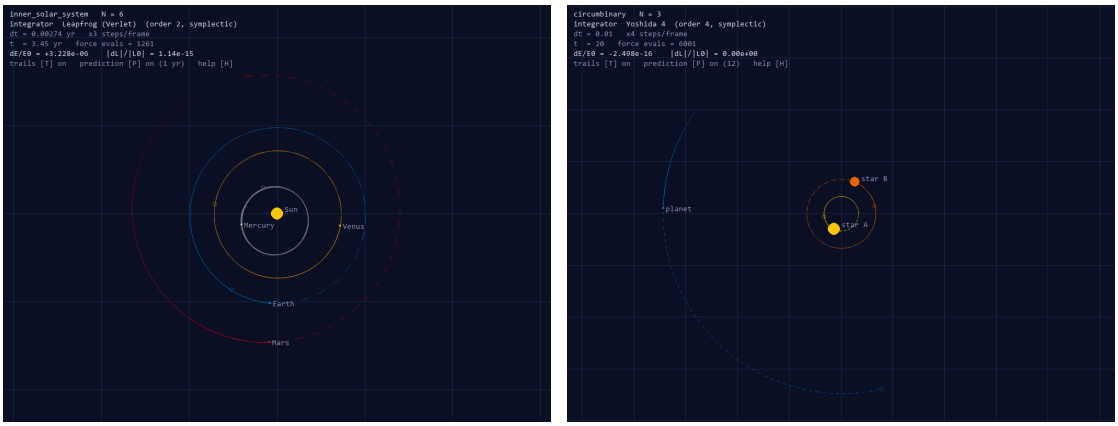


Figure 4.2: Viewer with trajectory overlays, rendered headless by experiment e09. Left: inner Solar System with leapfrog (h = 1 day); solid fading trails show the last 300 days, dotted lines the forecast for the next year. Right: planet around a binary star (Yoshida-4). The head-up display shows live conservation errors.

4.9 Testing and reproducibility

The test suite (`pytest`, 100 tests, about 30 s) is organised along the theory:

Table 4.1: Overview of the automated test suite.

Area	What is verified
Forces	two-body acceleration; dense kernel = pair loop = pair-list kernel = blocked kernel; $\sum_i m_i \boldsymbol{a}_i = 0$; $m_i \boldsymbol{a}_i = -\nabla_i V_\varepsilon$ by finite differences
Integrators	measured convergence order $= p \pm 0.35$ for all fixed-step methods; exact force-evaluation counts (FSAL); reversibility at round-off level iff symmetric; bounded energy error iff symplectic; $\det M = 1$ iff symplectic; conservation of $\boldsymbol{P}$ (all methods) and $\boldsymbol{L}$ (symplectic methods)
Adaptive	exact final time; step sizes vary by more than a factor of 20 on an $e = 0.9$ orbit; error decreases with the tolerance; evaluation count $1 + 6 \times$ (accepted + rejected steps)
Kepler solution	Kepler's equation solved to 10^{-12}; exact orbit conserves E, $\boldsymbol{L}$, $\boldsymbol{P}$; it satisfies the equations of motion and is periodic
Scenarios, I/O	all builtins valid and barycentric; virial ratio $1/2$; JSON round trip; original file format; `.npz` round trip; Solar System periods obey Kepler's third law
Overlays, metrics	ring-buffer order; forecast leaves the simulation untouched and equals the true future; order fit and Lyapunov fit on synthetic data

The experiments of Section 5 are Python modules (`experiments/e01_*.py` to `e09_*.py`). `python -m experiments.run_all` runs all of them in about six minutes on a laptop. They write SVG figures to `thesis/figures/generated` and CSV/JSON tables to `thesis/data`, which this document reads with Typst's `csv()` and `json()` functions. Random initial conditions use fixed seeds. The numerical results are therefore deterministic; only wall-clock timings depend on the machine.

4.10 Improvements over the original implementation

Table 4.2: Main differences between the original implementation [6] and the present one.

Original course project	**This work**
one Python object per body; double loop over pairs	structure-of-arrays state; vectorised kernels (Section 4.4)
leapfrog evaluated the forces twice per step	FSAL caching: one evaluation per step
energy diagnostic used the unsoftened potential with softened forces	consistent softened energy (Section 2.6)
Euler, leapfrog and Yoshida-4 without a common interface	nine methods behind a common interface; registry; generic RK and composition classes
mutable default arguments (`bodies=[]`, `integrator=LeapFrog()`) shared across instances	immutable defaults; integrators are stateless
physics coupled to screen coordinates (positions shifted to the window centre)	physics in physical units; camera transform only in the viewer
Sun initially at rest: centre-of-mass drift	barycentric initial conditions
`pickle` for stored runs	`.npz` without pickling
energy quotient $\lvert E_t/E_0 \rvert$ as the only metric	conservation, accuracy, structural, physical and cost metrics (Section 5.1)
no tests	100 automated tests derived from the theory

5 Evaluation

This chapter evaluates the integrators and the implementation on problems of increasing difficulty: a linear oscillator, an integrable two-body problem, a periodic three-body orbit, a chaotic three-body problem with close encounters, and the Solar System. It ends with a performance analysis. Section 5.1 defines the metrics; Section 5.2 to Section 5.9 present the experiments e01–e08, each implemented in `experiments/` and reproducible with one command.

5.1 Metrics and methodology

Conservation metrics. With $E(t)$ computed from the **same** (softened) Hamiltonian that generates the dynamics:

- relative energy error $\delta E(t) = (E(t) - E_0)/\ |E_0|$, its maximum $\max_t |\delta E|$ and its RMS;
- secular energy drift: the slope of the least-squares line through $\delta E(t)$. Unlike the maximum, it separates bounded oscillation from systematic growth;
- momentum error $\max_t |\boldsymbol{P}(t) - \boldsymbol{P}_0|\ /\ \sum_i m_i\ |\boldsymbol{v}_i(0)|$ (normalised because $\boldsymbol{P}_0 = \boldsymbol{0}$) and angular-momentum error $\max_t |\boldsymbol{L}(t) - \boldsymbol{L}_0|\ /\ |\boldsymbol{L}_0|$.

Accuracy metrics. Against a reference solution $\left(\boldsymbol{x}^{\mathrm{ref}}, \boldsymbol{v}^{\mathrm{ref}}\right)$:

- the phase-space error $e(t) = \left(|\boldsymbol{x} - \boldsymbol{x}^{\mathrm{ref}}|^2 + |\boldsymbol{v} - \boldsymbol{v}^{\mathrm{ref}}|^2\right)^{1/2}$, summed over all bodies, in units where the orbital length and velocity scales are one;
- the empirical order $\hat{p}$: the slope of a least-squares fit of $\log e$ against $\log h$, restricted to the asymptotic regime ($10^{-11} < e < 10^{-2}$) so that neither the round-off plateau nor the pre-asymptotic regime biases it;
- the error growth exponent β in $e(t) \propto t^{\beta}$, fitted over the last 90 % of a long run.

Structural metrics. The reversibility error $\max_i |\boldsymbol{x}_i - \boldsymbol{x}_i(0)|$ after n steps forward with h and n steps backward with $-h$.

Physical observables. Mean orbital periods, from a linear fit of the unwrapped polar angle (mean motion); finite-time Lyapunov exponents, from a fit of $\ln \delta(t)$.

Cost. The number of force evaluations, which is independent of hardware. Wherever methods are compared, they get the **same** evaluation budget, i.e. steps s times larger for a method with s evaluations per step.

References. The Kepler problem has the exact solution of Section 2.4. For the figure-eight, an independent eighth-order integrator (SciPy's DOP853 [27]) at $\mathrm{rtol} = \mathrm{atol} = 10^{-13}$ serves as reference; comparing it with a run at 10^{-12} shows that its own error is below $3.1 \cdot 10^{-7}$.

5.2 Linear stability and phase space

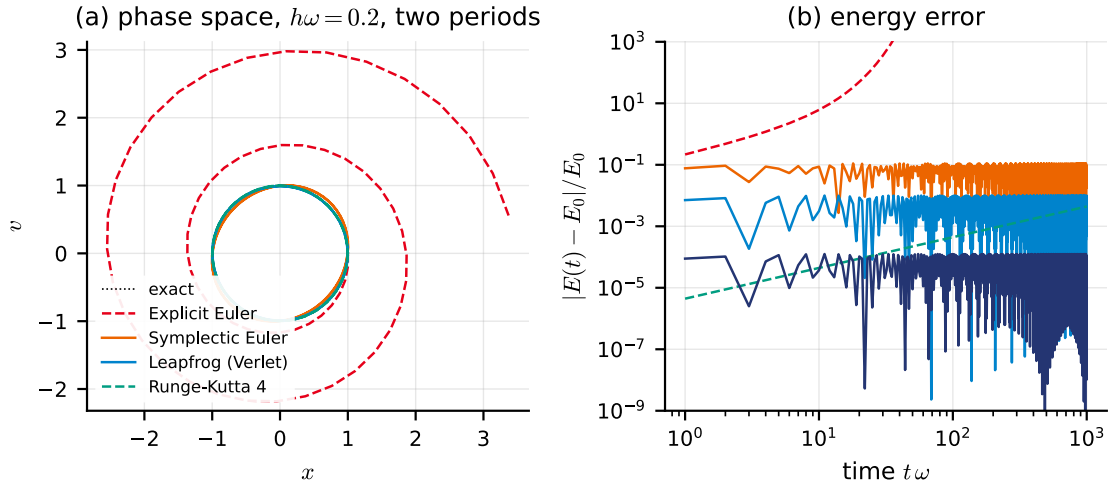


Figure 5.1: Harmonic oscillator with $h\omega = 0.2$. (a) Phase-space trajectories over two periods. (b) Relative energy error up to $t\omega = 1000$.

Figure 5.1 shows the behaviour predicted in Section 3. Explicit Euler spirals outwards: its energy grows by the factor $1 + h^2\omega^2$ per step and exceeds 10^3 after $t\omega \approx 100$. Symplectic Euler produces a slightly tilted ellipse, and its energy error oscillates with amplitude $\approx h\omega/2 = 0.1$, exactly as the conserved modified energy Equation 32 predicts. Leapfrog's error amplitude is $\approx (h\omega)^2/4 = 0.01$, as predicted by Equation 37, and Yoshida-4's is smaller still. None of them drifts. RK4, in contrast, **loses** energy steadily, at the rate

$(h\omega)^6/72 \approx 9 \cdot 10^{-7}$ per step derived in Section 3.6. Over the longest run this accumulates to $4 \cdot 10^{-3}$, more than Yoshida-4's bounded error although both are fourth order.

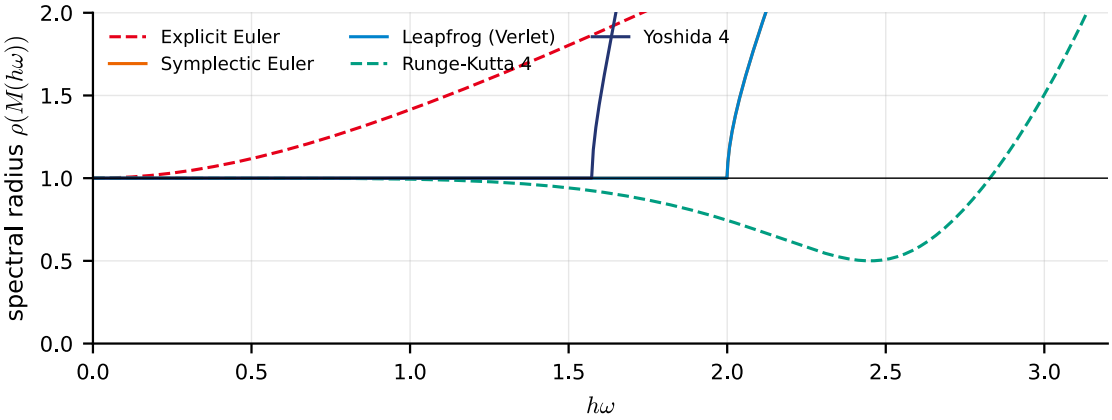


Figure 5.2: Spectral radius of the one-step matrix $M(h\omega)$. Symplectic Euler and leapfrog have identical eigenvalues ($\operatorname{tr} M = 2 - h^2\omega^2$ for both), so their curves coincide.

Figure 5.2 shows the spectral radius of the one-step matrices, computed numerically from the implementation itself. The measured stability limits are 2.002 (symplectic Euler, leapfrog; theory: 2, the deviation is the grid resolution), 2.829 (RK4; theory $2\sqrt{2} = 2.828$) and 1.575 (Yoshida-4); explicit Euler is unstable for every $h > 0$. Inside their stability intervals the symplectic methods have $\rho = 1$ exactly (no damping, no amplification), whereas RK4 has $\rho < 1$.

5.3 Convergence order and work-precision

Setup. Kepler orbit with $e = 0.6$, mass ratio 10^{-3}, $a = 1$, integrated over two periods with 2^4 to 2^{17} steps per period. The error is the maximum phase-space error over ≈ 256 equally spaced samples, measured against the exact solution.

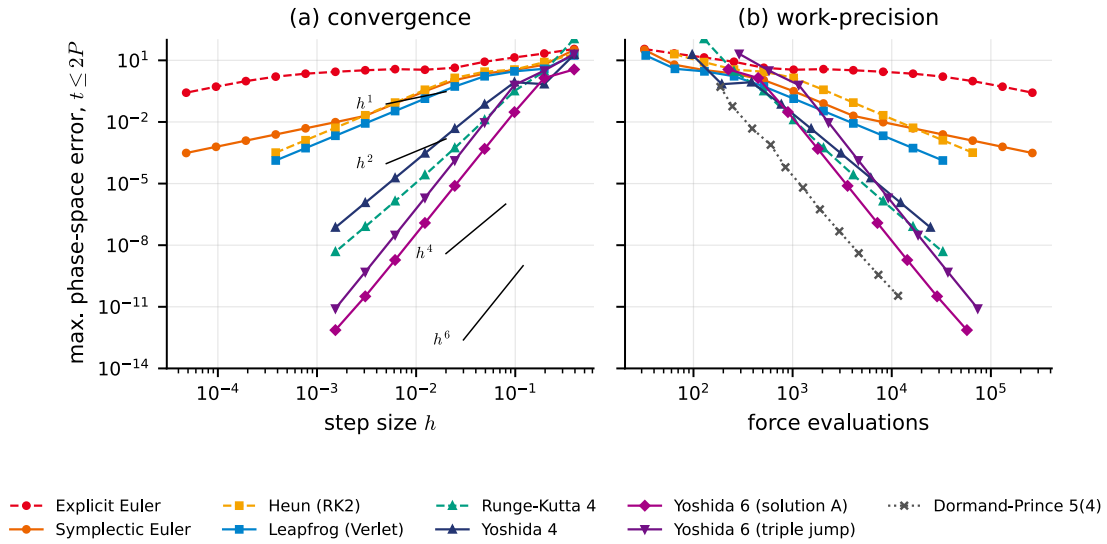


Figure 5.3: Kepler orbit ($e = 0.6$). (a) Maximum phase-space error versus step size; black lines indicate slopes h^p. (b) The same errors versus the number of force evaluations, including the adaptive Dormand–Prince method for tolerances 10^{-3} to 10^{-13}.

Table 5.1: Theoretical and measured convergence orders on the Kepler orbit with $e = 0.6$, and the number of force evaluations needed for a maximum phase-space error of 10^{-6} over two periods (interpolated; "not reached": requires more than the largest resolution tested).

Method	p (theory)	$\hat{p}$ (measured)	fit points	evals for $e = 10^{-6}$
Explicit Euler	1	0.94	3	not reached
Symplectic Euler	1	1.00	6	not reached
Heun (RK2)	2	2.01	3	not reached
Leapfrog (Verlet)	2	2.00	4	not reached
Runge-Kutta 4	4	4.19	5	8 875
Yoshida 4	4	4.00	5	12 758
Yoshida 6 (solution A)	6	5.97	5	5 043
Yoshida 6 (triple jump)	6	6.04	5	10 370
Dormand–Prince 5(4)	5	–	–	1 715

Orders. All measured orders agree with the theory to within 0.3 (Table 5.1). Explicit Euler reaches its asymptotic regime only for errors below $\mathcal{O}(1)$ and 2^{15} steps per period. Its order was therefore fitted in the window $e < 1$. This shows how slowly a first-order

method converges on an eccentric orbit. RK4 shows a slightly higher apparent order (≈ 4.2) because of pre-asymptotic contributions. As explained in Section 3.4, symplectic Euler converges with order one only if the error is measured over the whole interval; at the final time alone it would appear second order.

Work-precision. At equal cost the high-order methods are orders of magnitude more accurate (Figure 5.3 b). Two further observations matter for the choice of method:

1. **Over short times, symplecticity is not an advantage.** RK4 needs fewer evaluations than Yoshida-4 for $e = 10^{-6}$, because Yoshida-4 has a larger error constant (negative substep). The benefit of symplecticity lies in the long-time behaviour, studied next.
2. **Adaptivity pays off on eccentric orbits.** DOPRI5 concentrates its steps near periapsis and reaches 10^{-6} with the fewest evaluations of all methods, about a factor of 5.2 fewer than RK4. Among the fixed-step methods Yoshida-6A is cheapest; it beats the 9-stage triple-jump sixth-order method by about a factor of two at equal accuracy.

5.4 Long-term behaviour at equal cost

Setup. Kepler orbit with $e = 0.5$ over 200 periods. Every fixed-step method gets **1008 force evaluations per period**, i.e. 1008 leapfrog steps, 252 RK4 steps, 336 Yoshida-4 steps, and so on. DOPRI5 uses $\mathrm{rtol} = 10^{-10}$, which costs a similar 1327 evaluations per period.

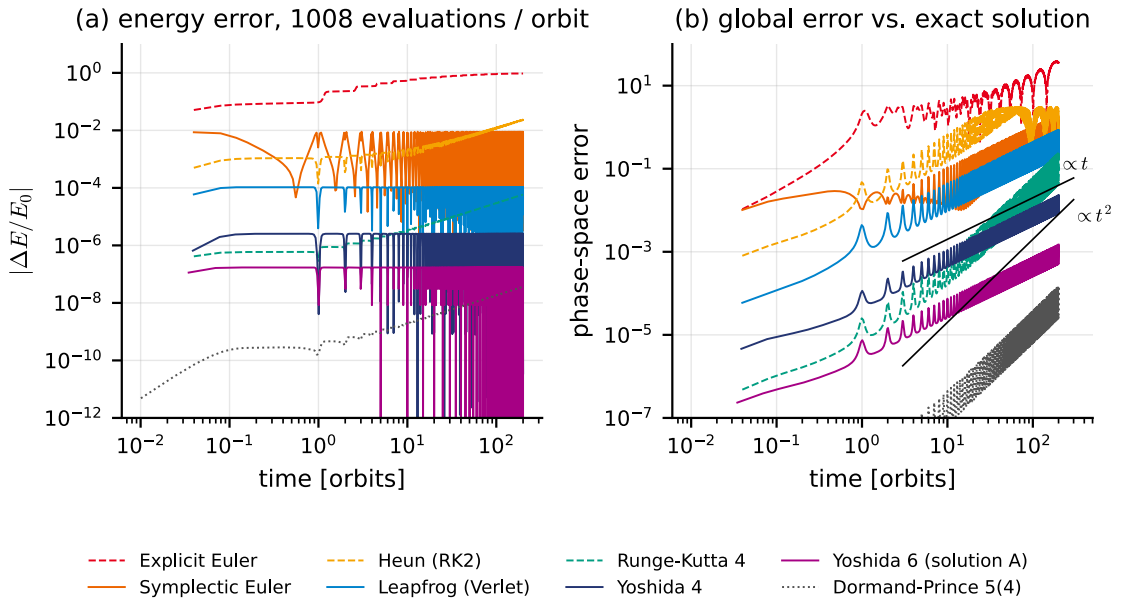


Figure 5.4: Kepler orbit ($e = 0.5$) over 200 periods at equal cost. (a) Relative energy error. (b) Phase-space error with respect to the exact solution; black lines indicate linear and quadratic growth.

Table 5.2: Long-term metrics for the experiment of Figure 5.4. "Drift / orbit": slope of $\delta E(t)$ per period. "Growth β": exponent of $e(t) \propto t^\beta$ (– where the error saturates at the orbit size). "Reversib.": position error after 1000 steps forward and backward with $h = P_{\mathrm{orb}}/200$.

Method	$\max\lvert\delta E\rvert$	**drift / orbit**	$\max\lvert\delta L\rvert$	$e(T)$	**growth** β	**reversib.**
Explicit Euler	$9.5 \cdot 10^{-1}$	$1.9 \cdot 10^{-3}$	$5.4 \cdot 10^{-1}$	$3.7 \cdot 10^{1}$	–	8.4
Symplectic Euler	$9.0 \cdot 10^{-3}$	$-2.4 \cdot 10^{-7}$	$1.8 \cdot 10^{-14}$	1.7	1.00	1.8
Heun (RK2)	$2.4 \cdot 10^{-2}$	$1.1 \cdot 10^{-4}$	$3.8 \cdot 10^{-3}$	2.5	–	$5.8 \cdot 10^{-1}$
Leapfrog (Verlet)	$1.1 \cdot 10^{-4}$	$4.5 \cdot 10^{-10}$	$3.1 \cdot 10^{-14}$	$8.4 \cdot 10^{-1}$	1.00	$9.5 \cdot 10^{-14}$
Runge-Kutta 4	$5.9 \cdot 10^{-5}$	$-2.9 \cdot 10^{-7}$	$9.0 \cdot 10^{-6}$	$2.4 \cdot 10^{-1}$	1.94	$3.8 \cdot 10^{-4}$
Yoshida 4	$2.5 \cdot 10^{-6}$	$1.0 \cdot 10^{-11}$	$2.6 \cdot 10^{-14}$	$2.3 \cdot 10^{-2}$	1.00	$3.8 \cdot 10^{-13}$
Yoshida 6 (solution A)	$1.7 \cdot 10^{-7}$	$8.2 \cdot 10^{-16}$	$5.5 \cdot 10^{-14}$	$1.5 \cdot 10^{-3}$	1.00	$7.2 \cdot 10^{-13}$
Dormand-Prince 5(4)	$3.6 \cdot 10^{-8}$	$1.8 \cdot 10^{-10}$	$1.7 \cdot 10^{-8}$	$1.5 \cdot 10^{-4}$	1.98	–

Figure 5.4 and Table 5.2 show the central result of this thesis.

Energy. All symplectic methods keep the energy error **bounded**: the error oscillates with every orbit but does not grow, and the fitted drift is at the level of noise. Explicit Euler and

Heun gain energy steadily; Euler's orbit is destroyed completely. RK4 and DOPRI5 drift linearly, $\delta E \propto t$, as predicted by backward error analysis.

Global error. The measured growth exponents are $\beta \approx 1.00$ for symplectic Euler, leapfrog, Yoshida-4 and Yoshida-6A, and $\beta \approx 2$ for RK4 and DOPRI5. This is exactly the linear-versus-quadratic law derived in Section 3.5. DOPRI5 is still the most accurate method after 200 orbits, thanks to its higher order and adaptive steps. Because its error grows quadratically, Yoshida-6A would overtake it after roughly $\left(e_{\mathrm{Y6A}(T)}/e_{\mathrm{DP}(T)}\right) \cdot T \approx 10 \cdot 200 = 2000$ orbits.

Invariants and reversibility. Symplectic methods conserve the angular momentum to round-off (10^{-14}). This is a property of all symplectic partitioned methods for rotationally invariant problems, since they conserve quadratic invariants of this form [4]. RK4 and DOPRI5 do not. The symmetric methods return to the initial state to 10^{-13} when integrated backwards; the non-symmetric ones do not.

5.5 Revisiting the original experiment

The original course project evaluated a two-body system ($G = 20$, $m = 1000$ and 10, softening $\varepsilon = 5$, $h = 0.1$, 5000 leapfrog steps) by the energy quotient $q(t) = |E_t/E_0|$, and observed periodic dips that it attributed to "large numerical errors when the satellite is strongly accelerated". The original energy diagnostic, however, used the **Newtonian** potential, while the dynamics used the **softened** force (Section 2.6).

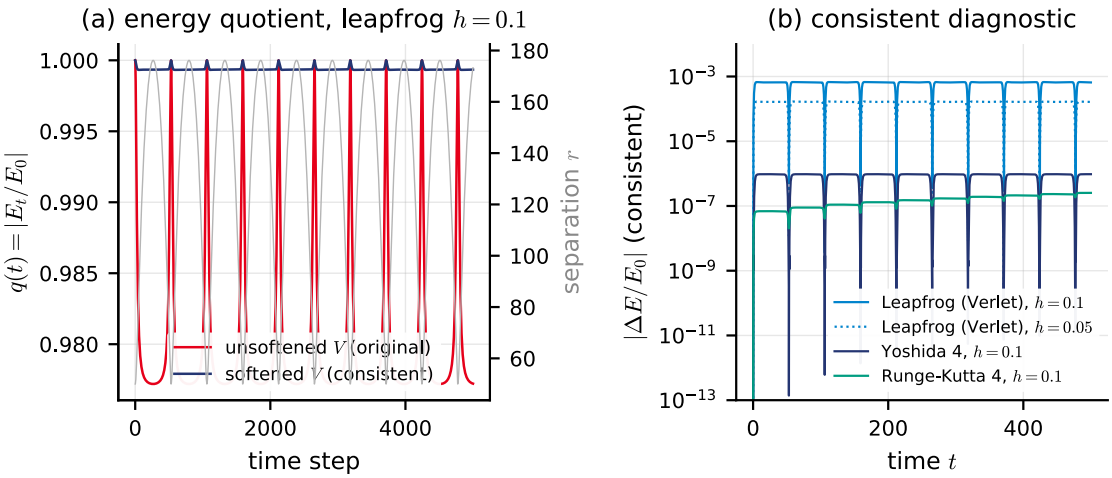


Figure 5.5: Original two-body experiment. (a) Energy quotient with the inconsistent diagnostic of the original project (red) and with the consistent softened energy (blue); grey: separation of the bodies. (b) Consistent relative energy error for several methods and step sizes.

With the consistent diagnostic the maximum deviation $\max|q - 1|$ drops from $2.3 \cdot 10^{-2}$ to $6.7 \cdot 10^{-4}$, a factor of 34 (Figure 5.5 a). About 97 % of the reported "energy error" was therefore an artefact of the diagnostic, not of the integrator. The remaining true error behaves as theory predicts: halving the step size reduces it by a factor of $4 \approx 2^2$, and the fourth-order methods reduce it to 10^{-7}–10^{-6} (Figure 5.5 b). This finding motivated the strict rule in the new code that energy diagnostics are always computed from the force model that generates the dynamics.

5.6 Periodic three-body orbit: the figure-eight

Setup. Figure-eight initial conditions [9], 50 periods, 1260 force evaluations per period for the fixed-step methods; DOPRI5 with $\mathrm{rtol} = 10^{-9}$. The reference is the independent DOP853 solution.

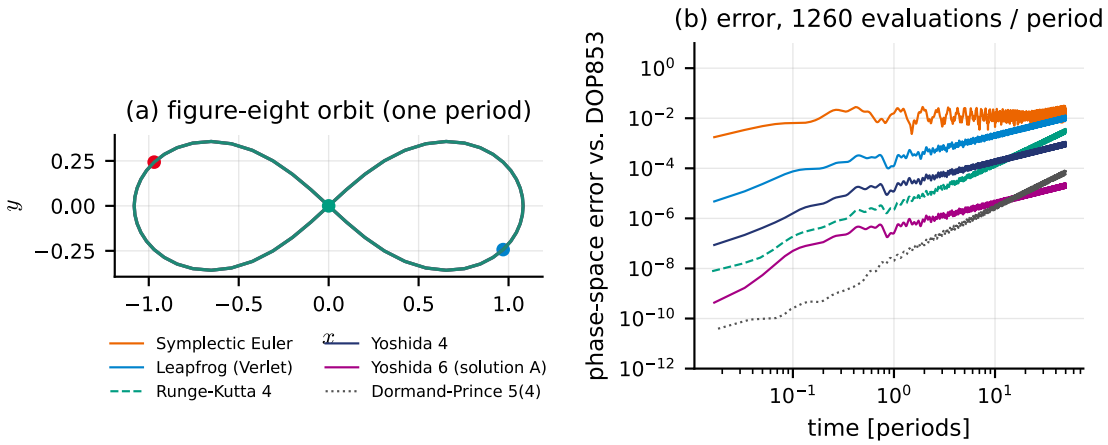


Figure 5.6: Figure-eight choreography. (a) The three bodies chase each other along the same curve. (b) Phase-space error with respect to the DOP853 reference.

Table 5.3: Figure-eight over 50 periods at (nearly) equal cost.

Method	**force evals**	$\max\lvert\delta E\rvert$	$\max\lvert\delta L\rvert$	$e(t = 50P)$
Symplectic Euler	63 000	$9.8 \cdot 10^{-4}$	$1.6 \cdot 10^{-14}$	$2.0 \cdot 10^{-2}$
Leapfrog (Verlet)	63 001	$1.5 \cdot 10^{-5}$	$1.6 \cdot 10^{-14}$	$8.3 \cdot 10^{-3}$
Runge-Kutta 4	63 000	$4.3 \cdot 10^{-6}$	$1.0 \cdot 10^{-8}$	$2.5 \cdot 10^{-3}$
Yoshida 4	63 001	$3.5 \cdot 10^{-8}$	$1.1 \cdot 10^{-14}$	$7.5 \cdot 10^{-4}$
Yoshida 6 (solution A)	63 001	$7.1 \cdot 10^{-9}$	$2.6 \cdot 10^{-14}$	$1.7 \cdot 10^{-5}$
Dormand-Prince 5(4)	99 907	$1.0 \cdot 10^{-7}$	$1.7 \cdot 10^{-10}$	$5.6 \cdot 10^{-5}$

The figure-eight is a stable periodic orbit, so errors grow polynomially, and higher order translates directly into higher accuracy (Table 5.3). At equal cost, Yoshida-6A is about 501 times more accurate than leapfrog and more accurate than DOPRI5, which used 60 % more evaluations. Again, only the symplectic methods conserve the angular momentum to machine precision.

5.7 Chaos and close encounters: the Pythagorean problem

Setup. Burrau's problem [10] (masses 3, 4, 5 at rest), $0 \le t \le 70$. DOPRI5 with $\mathrm{rtol} = 10^{-12}$ is compared with leapfrog and Yoshida-4 using the **same** number of force evaluations. A second DOPRI5 run with the position of one body perturbed by 10^{-8} measures

the sensitivity to initial conditions. A tolerance study compares the final states for $\mathrm{rtol} = 10^{-8}, \ldots, 10^{-12}$ with an independent SciPy DOP853 solution at $\mathrm{rtol} = \mathrm{atol} = 3 \cdot 10^{-14}$.

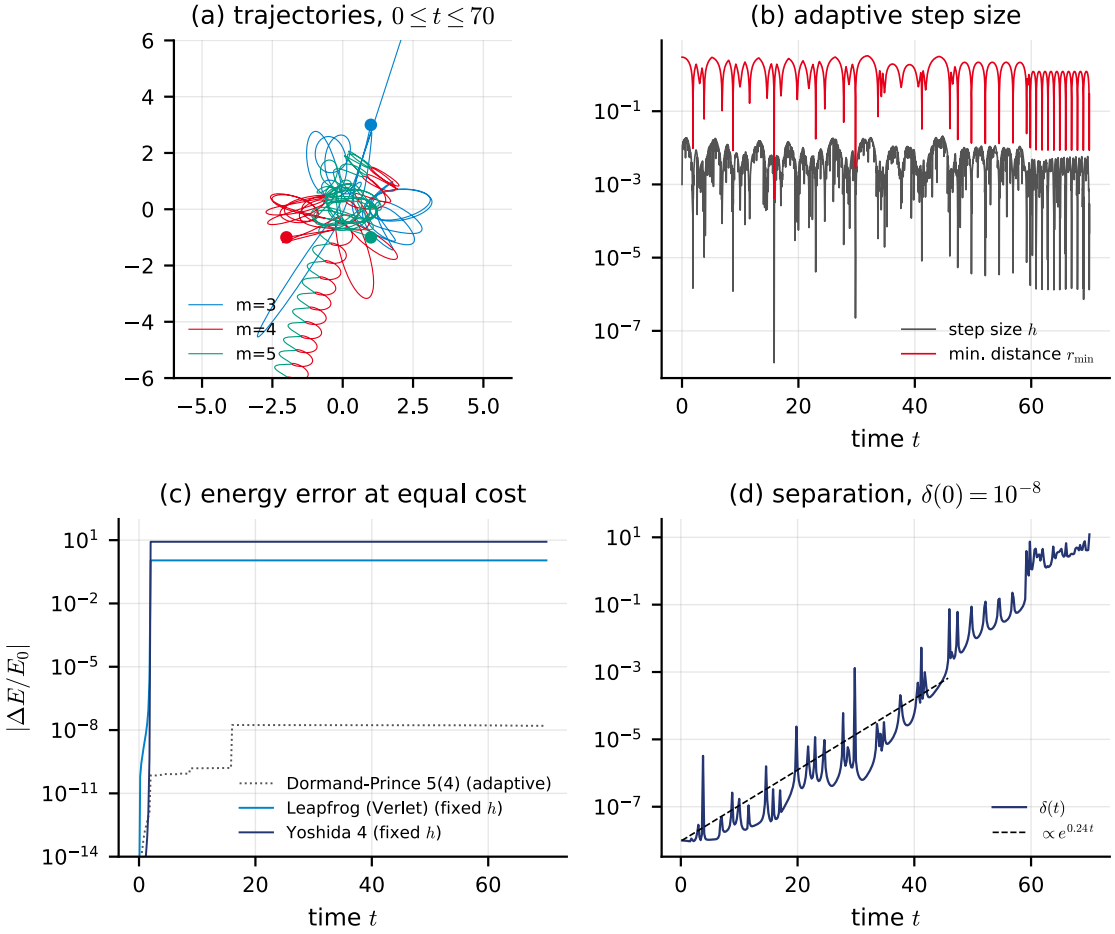


Figure 5.7: Pythagorean three-body problem (DOPRI5, $\mathrm{rtol} = 10^{-12}$). (a) Trajectories: after a sequence of close encounters the lightest body ($m = 3$) escapes and the two heavier bodies form a binary. (b) Adaptive step size and minimum pairwise distance. (c) Energy error at equal cost. (d) Phase-space separation of two runs whose initial conditions differ by 10^{-8}.

Adaptivity. The closest approach, $r_{\min} = 4.1 \cdot 10^{-4}$ at $t = 15.83$, forces DOPRI5 to reduce its step size to $1.3 \cdot 10^{-8}$. Elsewhere it takes steps up to $2.1 \cdot 10^{-2}$, so the step size varies over more than six orders of magnitude (Figure 5.7 b). The step size follows the minimum

distance closely; dimensionally, $h \propto r_{\min}^{3/2}$ is the local dynamical time. With 248 257 force evaluations (41 062 accepted and 314 rejected steps) the energy is conserved to $1.7 \cdot 10^{-8}$.

Fixed steps fail. With the same budget, leapfrog ($h = 2.8 \cdot 10^{-4}$) and Yoshida-4 ($h = 8.5 \cdot 10^{-4}$) cannot resolve the very first close encounter. The energy error jumps to $|\delta E| = 1.1$ and 8.4, respectively, and the trajectories become meaningless (Figure 5.7 c). Symplecticity does not help when the step size exceeds the time scale of the dynamics. This is precisely the regime in which the backward error expansion of Section 3.5 breaks down.

Chaos. The separation of the two adaptive runs grows exponentially, with large jumps at close encounters (Figure 5.7 d). A fit gives the finite-time Lyapunov exponent $\lambda \approx 0.24$. The separation reaches 10^{-2} at $t \approx 46$. Equation Equation 21 predicts $t_{\text{pred}} \approx \ln(10^{-2}/10^{-8})/\lambda \approx 57$, the right order of magnitude. It overestimates the horizon because the growth is concentrated in a few encounters rather than spread uniformly, as a single exponent assumes.

Table 5.4: Pythagorean problem: final state at $t = 70$ as a function of the DOPRI5 tolerance. Reference: SciPy DOP853 at $3 \cdot 10^{-14}$, which yields the binary m=4 + m=5. "Outcome": the same pair forms the final binary as in the reference.

rtol	force evals	final binary	outcome	final error vs. reference
$1 \cdot 10^{-8}$	32 467	m=3 + m=5	no	$2.5 \cdot 10^{1}$
$1 \cdot 10^{-9}$	48 055	m=4 + m=5	yes	$2.4 \cdot 10^{1}$
$1 \cdot 10^{-10}$	87 331	m=3 + m=5	no	$2.8 \cdot 10^{1}$
$1 \cdot 10^{-11}$	199 351	m=4 + m=5	yes	7.8
$1 \cdot 10^{-12}$	248 257	m=4 + m=5	yes	6.0

Which body escapes? Table 5.4 makes the consequence of chaos explicit. With $\mathrm{rtol} \geq 10^{-10}$ the **qualitative** outcome, i.e. which body is ejected, is essentially random: 10^{-9} happens to agree with the reference, while 10^{-8} and 10^{-10} eject the wrong body. Only from 10^{-11} on do all runs agree with the reference and with the literature [11]: the bodies with $m = 4$ and $m = 5$ form the binary and $m = 3$ escapes. Even then the final positions still differ from the reference by $\mathcal{O}(1)$, because small differences in the escape velocity accumulate linearly over the long escape. No integrator can deliver pointwise accuracy beyond the predictability horizon; only conserved quantities and robust qualitative features remain meaningful measures of accuracy there.

5.8 The Solar System over 1000 years

Setup. Sun and eight planets (simplified coplanar model, Section 4.7), 1000 years, with a budget of one force evaluation per day: leapfrog with $h = 1$ d, Yoshida-4 with $h = 3$ d, RK4 with $h = 4$ d.

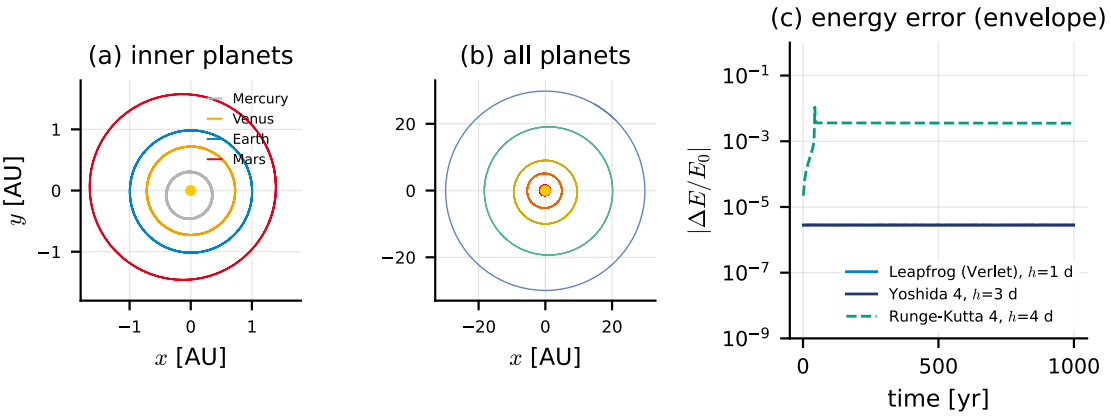


Figure 5.8: Solar System. (a) Inner planets over four years and (b) all planets over 170 years (heliocentric, leapfrog). (c) Relative energy error over 1000 years at one force evaluation per day.

Table 5.5: Conservation over 1000 years of Solar System evolution (365 251 force evaluations per run).

Method	h [d]	$\max\lvert\delta E\rvert$	drift / yr	$\max\lvert\delta L\rvert$	wall time
Leapfrog (Verlet)	1	$2.9 \cdot 10^{-6}$	$-3.2 \cdot 10^{-13}$	$6.0 \cdot 10^{-14}$	30 s
Yoshida 4	3	$2.8 \cdot 10^{-6}$	$-1.5 \cdot 10^{-13}$	$4.5 \cdot 10^{-14}$	31 s
Runge-Kutta 4	4	$1.1 \cdot 10^{-2}$	$8.9 \cdot 10^{-7}$	$2.3 \cdot 10^{-5}$	38 s

Conservation. Leapfrog and Yoshida-4 conserve the energy to $3 \cdot 10^{-6}$ over 1000 years without measurable drift, and the angular momentum to 10^{-13} (Table 5.5). RK4 at the same cost takes only 22 steps per Mercury orbit. Its numerical damping lets Mercury spiral inwards over a few decades until a close passage to the Sun destroys the orbit, and the energy error jumps to 10^{-2} (Figure 5.8 c). The details of this catastrophe depend sensitively on the step size: in a 100-year run with a marginally different step, Mercury was ejected from the Solar System.

Table 5.6: Simulated mean orbital periods compared with Kepler's third law Equation 16 (including the planet's own mass). Terrestrial planets: Yoshida-6A, h = 1 d, 100 years; giant planets: leapfrog, h = 1 d, 1000 years (more than 4000 steps per orbit).

Planet	a [AU]	P_{Kepler} [yr]	P_{sim} [yr]	rel. deviation
Mercury	0.387	0.2408	0.2408	$1.2 \cdot 10^{-6}$
Venus	0.723	0.6152	0.6152	$4.8 \cdot 10^{-6}$
Earth	1.000	1.0000	1.0000	$6.0 \cdot 10^{-6}$
Mars	1.524	1.8809	1.8809	$5.0 \cdot 10^{-5}$
Jupiter	5.203	11.8620	11.8651	$2.6 \cdot 10^{-4}$
Saturn	9.537	29.4465	29.3169	$-4.4 \cdot 10^{-3}$
Uranus	19.189	84.0571	84.2996	$2.9 \cdot 10^{-3}$
Neptune	30.070	164.8870	163.1850	$-1.0 \cdot 10^{-2}$

Physical validation. The simulated periods of the terrestrial planets agree with Kepler's third law to better than 10^{-4} (Table 5.6). The giant planets deviate by up to 1 %. This is physics rather than numerical error, as the convergence of the terrestrial periods shows. The two-body law ignores the mutual perturbations of the planets; these are strongest for Saturn, owing to the near 5:2 resonance with Jupiter ("great inequality"), and for Uranus and Neptune, owing to their near 2:1 commensurability [29]. In addition, the mass enclosed by the outer orbits includes the inner giant planets ($\approx 1.3 \cdot 10^{-3} M_{\odot}$). Finally, the artificial initial condition, with every planet at perihelion simultaneously, does not correspond to the planets' mean orbits.

Table 5.7: Relative period error of the equal-cost runs with respect to the converged reference (first 100 years).

Planet	P_{ref} [yr]	leapfrog (1 d)	Yoshida 4 (3 d)	RK4 (4 d)
Mercury	0.2408	$3.5 \cdot 10^{-3}$	$-5.1 \cdot 10^{-3}$	$8.4 \cdot 10^{-1}$
Venus	0.6152	$2.6 \cdot 10^{-4}$	$-4.6 \cdot 10^{-5}$	$-3.8 \cdot 10^{-4}$
Earth	1.0000	$1.0 \cdot 10^{-4}$	$-6.8 \cdot 10^{-6}$	$-2.0 \cdot 10^{-5}$
Mars	1.8809	$3.6 \cdot 10^{-5}$	$-5.4 \cdot 10^{-7}$	$-1.1 \cdot 10^{-6}$

Frequency errors. Energy conservation does not imply phase accuracy. Table 5.7 shows that Mercury's period is off by $3.5 \cdot 10^{-3}$ with leapfrog and $-5.1 \cdot 10^{-3}$ with Yoshida-4, although both conserve the energy to 10^{-6}. This is the frequency error $\delta n_{\mathrm{num}} = \mathcal{O}(h^{p})$ of Equation 36. It is invisible in the energy but makes the phase error grow linearly. It decreases rapidly for the outer planets, which are resolved with many more steps per orbit. For RK4 Mercury's period is wrong by 84 %, because the orbit has decayed.

5.9 Performance

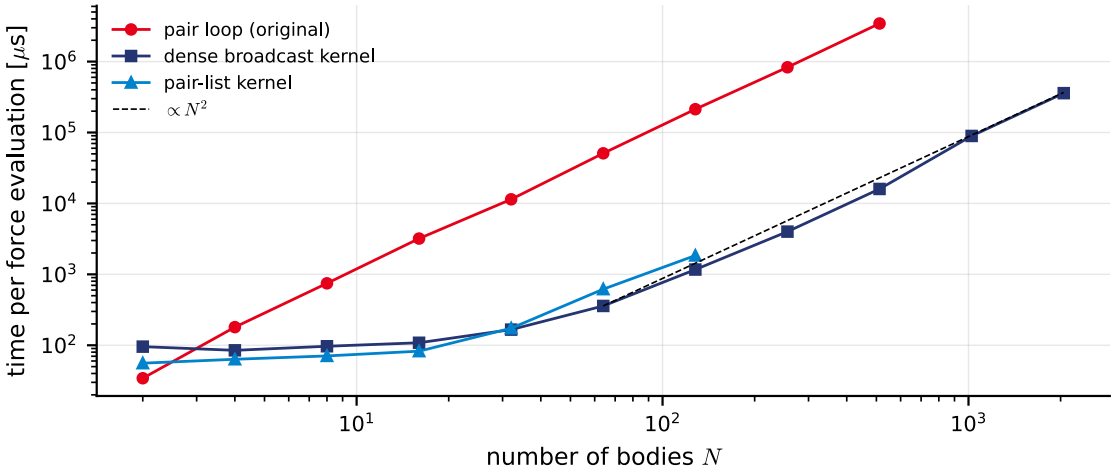


Figure 5.9: Wall-clock time per force evaluation versus N (Intel64 Family 6 Model 158 Stepping 13, GenuineIntel, Python 3.11.9, NumPy 2.3.1).

Figure 5.9 compares the pair loop of the original implementation with the two vectorised kernels. All three scale as $\mathcal{O}(N^2)$ for large N, but the vectorised kernels are faster by a factor of ≈ 182 at $N = 128$ and ≈ 215 at $N = 512$. For $N = 2$ the plain loop is actually faster (one pair; NumPy call overhead dominates). The pair-list kernel is the fastest choice up to $N \approx 30$ and the dense kernel above, which is why `GravityModel` switches between them automatically at $N = 24$. Together with the halved number of force evaluations of the cached leapfrog, a leapfrog step is faster than in the original implementation by a factor of ≈ 365 at $N = 128$. With $359\ \mathrm{ms}$ per evaluation at $N = 2048$, direct summation in Python reaches its practical limit, which motivates the tree and multipole methods discussed in Section 6.

5.10 Discussion

The experiments confirm the theory of Section 3 quantitatively, and they also show its limits:

- The **order** of a method determines its accuracy per unit cost on smooth problems (Section 5.3, Section 5.6). Sixth-order composition methods are the most efficient fixed-step methods once the accuracy requirement exceeds about 10^{-6}.
- **Symplecticity** does not reduce the error over short times, but it changes how the error accumulates: bounded energy error, linear instead of quadratic phase-error growth, and exact conservation of angular momentum (Section 5.4, Section 5.8). For long integrations this outweighs larger error constants.
- **Adaptivity** is indispensable when time scales vary strongly (Section 5.7). Combining it with symplecticity requires dedicated techniques (Section 6).
- **Correct diagnostics** are as important as correct integrators (Section 5.5). A metric that is not conserved by the exact dynamics cannot measure the quality of a numerical method.

6 Conclusion and Outlook

6.1 Summary

This thesis revisited the gravitational N-body problem of a computational physics course project and developed it into a systematic study of structure-preserving time integration.

On the **theoretical** side we derived the Hamiltonian structure of Newtonian gravity, its conservation laws and the exact solution of the Kepler problem. We then built a hierarchy of integrators of increasing sophistication, from explicit Euler through symplectic Euler, leapfrog, Heun and RK4 to Yoshida's composition methods of order four and six and an adaptive Dormand–Prince pair. For each method we derived the order, the linear stability limit and the presence or absence of symplecticity and time symmetry. Backward error analysis, in particular the modified Hamiltonian of symplectic methods, explains why the energy error of symplectic methods stays bounded. Combined with the energy dependence of the Kepler frequency, it predicts linear global-error growth for symplectic and quadratic growth for non-symplectic methods.

On the **implementation** side the original project was rebuilt as a modular package. It uses structure-of-arrays storage, vectorised force kernels, a common integrator interface with generic Runge–Kutta and composition building blocks, a registry, consistent energy diagnostics, a reproducible experiment pipeline, and an interactive viewer whose overlays show past trails and simulated future trajectories. One hundred automated tests check the implementation against the theory.

The **evaluation** confirmed the theory quantitatively:

- The measured convergence orders of all eight fixed-step methods agree with the theoretical ones to within 0.3.

- At equal cost, every symplectic method kept the energy error bounded over 200 eccentric orbits and showed linear error growth (measured exponent 1.00). RK4 and DOPRI5 showed quadratic growth (exponents near 2).
- The original project's energy fluctuation was mostly an artefact of an inconsistent energy diagnostic, not a weakness of leapfrog.
- High order pays off for smooth problems: Yoshida-6A was the most accurate fixed-step method at equal cost on the Kepler and figure-eight problems.
- Close encounters, as in the Pythagorean problem, require adaptive step sizes. Fixed-step methods failed completely at equal cost there. Because the problem is chaotic, even the qualitative outcome (which body escapes) is only reproduced reliably for tolerances of 10^{-11} and below.
- A 1000-year Solar System integration conserved the energy to $3 \cdot 10^{-6}$ at one force evaluation per day, while RK4 at the same cost destroyed Mercury's orbit. The simulated periods of the terrestrial planets agree with Kepler's third law to better than 10^{-4}.

6.2 Recommendations

The experiments suggest a simple guideline for choosing an integrator:

Table 6.1: Guideline for choosing an integrator, derived from the experiments.

Situation	**Recommended method**
quick, robust default; real-time visualisation	leapfrog (1 evaluation/step, symplectic, reversible)
long integrations of smooth systems, high accuracy	Yoshida 6A (or Yoshida 4 at moderate accuracy)
short integrations, moderate accuracy	RK4 or DOPRI5 are competitive
close encounters, widely varying time scales	adaptive DOPRI5 (or a regularised method, see below)
chaotic systems	any method that conserves the invariants; judge results statistically
teaching / never	explicit Euler

6.3 Limitations and outlook

The study deliberately covers general-purpose methods. Several directions would extend it in a natural way:

- **Planetary systems.** Wisdom–Holman mixed-variable symplectic maps [30] split the Hamiltonian into Keplerian motion around the star, which is integrated exactly with the solver of Section 2.4, and small planet–planet interactions. This permits steps of a sizeable fraction of the shortest orbital period. The `Composition` and `ForceModel` abstractions could host such a splitting directly.
- **Structure-preserving adaptivity.** Time transformations (Sundman, Poincaré) and the regularisations of Kustaanheimo–Stiefel or Burdet–Heggie make close encounters regular and keep the symplectic structure [1], [4]. IAS15 [31] shows that an adaptive 15th-order method can reach machine precision with errors that grow only as a random walk.
- **Large N.** The direct $\mathcal{O}(N^2)$ summation limits the package to a few thousand bodies. Tree codes [32] and the fast multipole method [33] reduce the cost to $\mathcal{O}(N \log N)$ or $\mathcal{O}(N)$. Just-in-time compilation or GPU kernels would remove the Python overhead that dominates for small N.
- **Physics.** Three-dimensional initial conditions with inclinations from full ephemerides would allow a quantitative comparison with observed planetary motion. A first post-Newtonian correction would reproduce the relativistic perihelion advance of Mercury of 43 arcseconds per century.
- **Visualisation.** The forecast overlay could show an ensemble of perturbed forecasts, making the loss of predictability in chaotic systems (Equation 21) directly visible.

A User Guide

A.1 Installation

```
pip install -e ".[viewer,experiments,dev]"   # from the repository root
python -m pytest                              # run the 100 tests
```

The package requires Python $\geq$ 3.10, NumPy and Matplotlib. pygame is needed only for the viewer, SciPy only for the experiments and tests.

A.2 Command-line interface

```
python -m nbody list                                  # scenarios and integrators
python -m nbody simulate kepler -i yoshida4 --dt 0.01 --t-end 100 --plot
python -m nbody simulate kepler -p e=0.9 mass_ratio=0.01 --out run.npz
python -m nbody compare figure_eight -i leapfrog rk4 yoshida4 yoshida6a
python -m nbody view solar_system -i leapfrog --steps-per-frame 4
python -m nbody view scenarios/three_body_legacy.json
```

In the viewer: `SPACE` pause, `T` trails, `P` forecast, `V` velocity vectors, `C` follow the centre of mass, `F` fit view, `+`/`-`/mouse wheel zoom, arrow keys pan, `.`/`,` faster/slower, `1`–`9` switch integrator, `R` reset, `S` screenshot, `H` help.

A.3 Python API

```
from nbody import load_scenario, simulate
from nbody.analysis import conservation_report

sc = load_scenario("kepler", e=0.6)
traj = simulate(sc.model(), sc.initial_state(), "yoshida4", t_end=100.0, dt=0.01)
print(conservation_report(traj))   # conservation errors
traj.save("kepler.npz")
```

A.4 Scenario file format

```
{
  "name": "binary",
  "description": "Equal-mass binary on a circular orbit",
  "G": 1.0,
  "softening": 0.0,
  "units": {"length": "", "time": "", "mass": ""},
  "recommended": {"integrator": "leapfrog", "dt": 0.01, "t_end": 50.0},
  "bodies": [
    {"name": "A", "mass": 0.5, "color": "#0083CC",
     "position": [-0.5, 0], "velocity": [0, -0.5]},
    {"name": "B", "mass": 0.5, "color": "#E6001A",
     "position": [0.5, 0], "velocity": [0, 0.5]}
  ]
}
```

All keys except `bodies` are optional, so the files of the original course project (which contain only `G` and `bodies`) remain valid. Positions and velocities may be two- or three-dimensional.

A.5 Reproducing the results

```
python scripts/fetch_fonts.py               # template fonts
python -m experiments.run_all               # all experiments (~6 min)
typst compile thesis/main.typ --font-path thesis/assets/fonts
```

B Method Coefficients

B.1 Composition weights

All composition methods apply the leapfrog step Equation 34 with step sizes $\gamma_k h$, $k = 1, \dots, s$. The weights are palindromic and sum to one.

Weights of the symplectic composition methods.

Method	s	Weights $\gamma_1, \dots, \gamma_s$
Yoshida 4 (triple jump of leapfrog)	3	$\gamma_1 = \gamma_3 = 1/\left(2 - 2^{1/3}\right) = 1.351207191959658$ $\gamma_2 = -2^{1/3}/\left(2 - 2^{1/3}\right) = -1.702414383919315$
Yoshida 6 (triple jump of Yoshida 4)	9	$\gamma_{3(i-1)+j} = \beta_i \alpha_j$, α = Yoshida-4 weights, $\beta_1 = \beta_3 = 1/\left(2 - 2^{1/5}\right) = 1.174671758089363$, $\beta_2 = -2^{1/5}/\left(2 - 2^{1/5}\right) = -1.349343516178726$
Yoshida 6, solution A [20]	7	$(w_3, w_2, w_1, w_0, w_1, w_2, w_3)$ with $w_1 = -1.17767998417887$, $w_2 = 0.235573213359357$, $w_3 = 0.784513610477560$, $w_0 = 1 - 2(w_1 + w_2 + w_3)$

B.2 Dormand–Prince 5(4)

The Butcher tableau of DOPRI5(4) [24]. The row b gives the fifth-order solution, which is propagated; it equals the last row of A (FSAL). The row $\hat{b}$ gives the embedded fourth-order solution used for the error estimate.

Butcher tableau of the Dormand–Prince 5(4) pair.

$$
\begin{array}{r|cccccc}
0 & & & & & & \\
1/5 & 1/5 & & & & & \\
\end{array}
$$

3/10	3/40	9/40					
4/5	44/45	−56/15	32/9				
8/9	19372/6561	−25360/2187	64448/6561	−212/729			
1	9017/3168	−355/33	46732/5247	49/176	−5103/18656		
1	35/384	0	500/1113	125/192	−2187/6784	11/84	
b	35/384	0	500/1113	125/192	−2187/6784	11/84	0
$\hat{b}$	5179/57600	0	7571/16695	393/640	−92097/339200	187/2100	1/40

Bibliography

[1] S. J. Aarseth, *Gravitational N-Body Simulations: Tools and Algorithms*. Cambridge: Cambridge University Press, 2003. doi: 10.1017/CBO9780511535246.

[2] H. Poincaré, "Sur le problème des trois corps et les équations de la dynamique," *Acta Mathematica*, vol. 13, pp. 1–270, 1890, doi: 10.1007/BF02392506.

[3] K. F. Sundman, "Mémoire sur le problème des trois corps," *Acta Mathematica*, vol. 36, pp. 105–179, 1913, doi: 10.1007/BF02422379.

[4] E. Hairer, C. Lubich, and G. Wanner, *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*, 2nd ed., vol. 31. in Springer Series in Computational Mathematics, vol. 31. Berlin, Heidelberg: Springer, 2006. doi: 10.1007/3-540-30666-8.

[5] J. M. Sanz-Serna and M.-P. Calvo, *Numerical Hamiltonian Problems*. London: Chapman & Hall, 1994.

[6] C. El Aaouadi, E. Can, and S. Pfeiffer, "Computerphysik WiSe 24/25 – N-Körper-Problem." [Online]. Available: https://github.com/erencan-02/Computerphysik-Python

[7] H. Goldstein, C. P. Poole, and J. L. Safko, *Classical Mechanics*, 3rd ed. San Francisco: Addison-Wesley, 2002.

[8] E. Noether, "Invariante Variationsprobleme," *Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen, Mathematisch-Physikalische Klasse*, pp. 235–257, 1918.

[9] A. Chenciner and R. Montgomery, "A remarkable periodic solution of the three-body problem in the case of equal masses," *Annals of Mathematics*, vol. 152, no. 3, pp. 881–901, 2000, doi: 10.2307/2661357.

[10] C. Burrau, “Numerische Berechnung eines Spezialfalles des Dreikörperproblems,” *Astronomische Nachrichten*, vol. 195, no. 6, pp. 113–118, 1913, doi: 10.1002/asna.19131950602.

[11] V. Szebehely and C. F. Peters, “Complete solution of a general problem of three bodies,” *The Astronomical Journal*, vol. 72, pp. 876–883, 1967, doi: 10.1086/110355.

[12] H. C. Plummer, “On the problem of distribution in globular star clusters,” *Monthly Notices of the Royal Astronomical Society*, vol. 71, pp. 460–470, 1911, doi: 10.1093/mnras/71.5.460.

[13] H. Das, S. Deb, and A. Baruah, “Optimal softening for gravitational force calculations in N-body dynamics,” *The Astrophysical Journal*, vol. 911, no. 2, p. 83, 2021.

[14] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed., vol. 8. in Springer Series in Computational Mathematics, vol. 8. Berlin, Heidelberg: Springer, 1993. doi: 10.1007/978-3-540-78862-1.

[15] G. Strang, “On the construction and comparison of difference schemes,” *SIAM Journal on Numerical Analysis*, vol. 5, no. 3, pp. 506–517, 1968, doi: 10.1137/0705041.

[16] L. Verlet, “Computer experiments on classical fluids. I. Thermodynamical properties of Lennard-Jones molecules,” *Physical Review*, vol. 159, no. 1, pp. 98–103, 1967, doi: 10.1103/PhysRev.159.98.

[17] G. Benettin and A. Giorgilli, “On the Hamiltonian interpolation of near-to-the-identity symplectic mappings with application to symplectic integration algorithms,” *Journal of Statistical Physics*, vol. 74, no. 5, pp. 1117–1143, 1994, doi: 10.1007/BF02188219.

[18] M.-P. Calvo and J. M. Sanz-Serna, “The development of variable-step symplectic integrators, with application to the two-body problem,” *SIAM Journal on Scientific Computing*, vol. 14, no. 4, pp. 936–952, 1993, doi: 10.1137/0914057.

[19] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. Chichester: Wiley, 2016. doi: 10.1002/9781119121534.

[20] H. Yoshida, “Construction of higher order symplectic integrators,” *Physics Letters A*, vol. 150, no. 5–7, pp. 262–268, 1990, doi: 10.1016/0375-9601(90)90092-3.

[21] M. Suzuki, “Fractal decomposition of exponential operators with applications to many-body theories and Monte Carlo simulations,” *Physics Letters A*, vol. 146, no. 6, pp. 319–323, 1990, doi: 10.1016/0375-9601(90)90962-N.

[22] M. Creutz and A. Gocksch, “Higher-order hybrid Monte Carlo algorithms,” *Physical Review Letters*, vol. 63, no. 1, pp. 9–12, 1989, doi: 10.1103/PhysRevLett.63.9.

[23] E. Forest and R. D. Ruth, “Fourth-order symplectic integration,” *Physica D: Nonlinear Phenomena*, vol. 43, no. 1, pp. 105–117, 1990, doi: 10.1016/0167-2789(90)90019-L.

[24] J. R. Dormand and P. J. Prince, “A family of embedded Runge–Kutta formulae,” *Journal of Computational and Applied Mathematics*, vol. 6, no. 1, pp. 19–26, 1980, doi: 10.1016/0771-050X(80)90013-3.

[25] C. R. Harris, K. J. Millman, S. J. van der Walt, and others, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[26] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.

[27] P. Virtanen, R. Gommers, T. E. Oliphant, and others, “SciPy 1.0: fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[28] E. M. Standish and J. G. Williams, “Keplerian Elements for Approximate Positions of the Major Planets.” [Online]. Available: https://ssd.jpl.nasa.gov/planets/approx_pos.html

[29] C. D. Murray and S. F. Dermott, *Solar System Dynamics*. Cambridge: Cambridge University Press, 1999. doi: 10.1017/CBO9781139174817.

[30] J. Wisdom and M. Holman, “Symplectic maps for the n-body problem,” *The Astronomical Journal*, vol. 102, pp. 1528–1538, 1991, doi: 10.1086/115978.

[31] H. Rein and D. S. Spiegel, “IAS15: a fast, adaptive, high-order integrator for gravitational dynamics, accurate to machine precision over a billion orbits,” *Monthly Notices of the Royal Astronomical Society*, vol. 446, no. 2, pp. 1424–1437, 2015, doi: 10.1093/mnras/stu2164.

[32] J. Barnes and P. Hut, “A hierarchical O(N log N) force-calculation algorithm,” *Nature*, vol. 324, pp. 446–449, 1986, doi: 10.1038/324446a0.

[33] L. Greengard and V. Rokhlin, “A fast algorithm for particle simulations,” *Journal of Computational Physics*, vol. 73, no. 2, pp. 325–348, 1987, doi: 10.1016/0021-9991(87)90140-9.